



ESTRUTURAS DE DADOS

EXERCÍCIOS DE IMPLEMENTAÇÃO, CONCEITO E ALGORITMOS

LISTA DE EXERCÍCIOS

BALANÇO TÉCNICO

TIPOS ABSTRATOS DE DADOS

- ① Implemente um TAD **Ponto** que represente um ponto no plano cartesiano com coordenadas (x, y).

O TAD deve fornecer as seguintes operações:

- Criar um ponto com coordenadas x e y
- Obter as coordenadas x e y
- Calcular a distância do ponto até a origem (0, 0)
- Calcular a distância entre dois pontos
- Exibir o ponto no formato “(x, y)”

Exemplo de uso

```
Ponto p1 = criarPonto(3, 4);  
Ponto p2 = criarPonto(0, 0);
```

```
Distância até origem: 5.00  
Distância entre p1 e p2: 5.00  
Coordenadas: (3, 4)
```

- ② Implemente um TAD **Data** que represente uma data com dia, mês e ano.

O TAD deve fornecer as seguintes operações:

- Criar uma data (com validação de data válida)
- Verificar se o ano é bissexto
- Comparar duas datas (anterior, igual ou posterior)
- Calcular a diferença em dias entre duas datas
- Adicionar N dias a uma data
- Exibir a data no formato “DD/MM/AAAA”

Exemplo de uso

```
Data d1 = criarData(15, 3, 2024);  
Data d2 = criarData(20, 3, 2024);
```

```
Data válida: true  
Bissexto: true  
Comparação: d1 é anterior a d2  
Diferença: 5 dias  
d1 + 10 dias: 25/03/2024
```

- 3 Implemente um TAD Conjunto que represente um conjunto matemático de inteiros.

O TAD deve fornecer as seguintes operações:

- Criar conjunto vazio
- Inserir elemento (sem repetições)
- Remover elemento
- Verificar se elemento pertence ao conjunto
- Obter tamanho do conjunto
- União de dois conjuntos
- Interseção de dois conjuntos
- Diferença entre dois conjuntos
- Verificar se um conjunto é subconjunto de outro
- Exibir elementos do conjunto

Exemplo de uso

Conjunto A = {1, 2, 3, 4}

Conjunto B = {3, 4, 5, 6}

$A \cup B = \{1, 2, 3, 4, 5, 6\}$

$A \cap B = \{3, 4\}$

$A - B = \{1, 2\}$

$3 \in A$: true

$A \subseteq B$: false

- 4 Implemente um TAD **MatrizEsparsa** que represente uma matriz esparsa (com muitos elementos zero) de forma eficiente.

O TAD deve armazenar apenas elementos não-nulos usando lista encadeada e fornecer:

- Criar matriz esparsa com dimensões $M \times N$
- Inserir elemento em posição (i, j)
- Acessar elemento em posição (i, j)
- Somar duas matrizes esparsas
- Multiplicar duas matrizes esparsas
- Transpor a matriz
- Exibir matriz completa (incluindo zeros)

Exemplo de uso

```
MatrizEsparsa M1(5, 5);  
M1.inserir(0, 0, 10);  
M1.inserir(2, 3, 20);  
M1.inserir(4, 4, 30);
```

Elementos armazenados: 3 (de 25 possíveis)

$M1[2][3] = 20$

$M1[1][1] = 0$

Matriz transposta:

```
10  0  0  0  0  
 0  0  0  0  0  
 0  0  0 20  0  
 0  0  0  0  0  
 0  0  0  0 30
```

LISTAS E ARRAYS

- 5 Crie um programa em C++ que implemente uma lista encadeada simples (singly linked list) e exiba seus elementos. O programa deve permitir ao usuário inserir valores na lista e, ao final, exibir todos os elementos na ordem em que foram inseridos.

Exemplo de entrada

```
Digite a quantidade de elementos: 6
Digite o elemento 1: 11
Digite o elemento 2: 9
Digite o elemento 3: 7
Digite o elemento 4: 5
Digite o elemento 5: 3
Digite o elemento 6: 1
```

Exemplo de saída

```
Lista encadeada: 11 9 7 5 3 1
```

- 6 Crie um programa em C++ que crie uma lista encadeada simples de n nós e exiba seus elementos em ordem reversa. O programa deve solicitar ao usuário a quantidade de elementos e seus valores, e depois exibir a lista do último elemento para o primeiro.

Exemplo de entrada

```
Digite a quantidade de elementos: 6
Digite o elemento 1: 11
Digite o elemento 2: 9
Digite o elemento 3: 7
Digite o elemento 4: 5
Digite o elemento 5: 3
Digite o elemento 6: 1
```

Exemplo de saída

```
Lista original: 11 9 7 5 3 1
Lista reversa: 1 3 5 7 9 11
```

- 7 Crie um programa em C++ que crie uma lista encadeada simples de n nós e conte o número total de nós. O programa deve permitir inserir elementos na lista e, ao final, exibir a quantidade total de nós presentes.

Exemplo de entrada

```
Digite a quantidade de elementos: 7
Digite o elemento 1: 13
Digite o elemento 2: 11
Digite o elemento 3: 9
Digite o elemento 4: 7
Digite o elemento 5: 5
Digite o elemento 6: 3
Digite o elemento 7: 1
```

Exemplo de saída

```
Lista encadeada: 13 11 9 7 5 3 1
Número de nós na lista: 7
```

- 8 Crie um programa em C++ que insira um novo nó no início de uma lista encadeada simples. O programa deve criar uma lista com alguns elementos, solicitar um novo valor ao usuário e inseri-lo no início da lista.

Exemplo de entrada

```
Lista original: 13 11 9 7 5 3 1
Digite o valor a ser inserido no início: 0
```

Exemplo de saída

```
Lista após inserção no início: 0 13 11 9 7 5 3 1
```

- 9 Crie um programa em C++ que encontre o elemento do meio de uma lista encadeada. O programa deve criar uma lista e determinar qual elemento está na posição central. Se a lista tiver número par de elementos, retorne o segundo elemento do meio.

Pseudo-código

```
slow = head
fast = head
enquanto fast ≠ null e fast.next ≠ null:
    slow = slow.next
    fast = fast.next.next
retornar slow.valor
```

Exemplo de entrada 1

Lista: 7 5 3 1

Exemplo de saída 1

Elemento do meio: 3

Exemplo de entrada 2

Lista: 9 7 5 3 1

Exemplo de saída 2

Elemento do meio: 5

- 10 Crie um programa em C++ que insira um novo nó no meio de uma lista encadeada simples. O programa deve criar uma lista, calcular a posição do meio e inserir o novo valor nessa posição.

Exemplo de entrada

Lista original: 7 5 3 1
Digite o valor a ser inserido no meio: 9

Exemplo de saída (após primeira inserção)

Lista após inserção: 7 5 9 3 1

Exemplo de saída (após segunda inserção)

Lista após inserção: 7 5 9 11 3 1

- 11 Crie um programa em C++ que obtenha o n-ésimo nó de uma lista encadeada simples. O programa deve criar uma lista, solicitar ao usuário uma posição e retornar o valor do nó nessa posição (1-indexed).

Exemplo de entrada

Lista: 7 5 3 1
Digite a posição desejada: 3

Exemplo de saída

Valor na posição 3: 3

- 12 Crie um programa em C++ que insira um novo nó em qualquer posição de uma lista encadeada simples. O programa deve criar uma lista, solicitar ao usuário a posição e o valor a ser inserido, e inserir o nó na posição especificada (1-indexed).

Exemplo de entrada 1

Lista original: 7 5 3 1
Digite a posição: 1
Digite o valor: 12

Exemplo de saída 1

Lista atualizada: 12 7 5 3 1

Exemplo de entrada 2

Lista atual: 12 7 5 3 1
Digite a posição: 4
Digite o valor: 14

Exemplo de saída 2

Lista atualizada: 12 7 5 14 3 1

- 13 Crie um programa em C++ que delete o primeiro nó de uma lista encadeada simples. O programa deve criar uma lista, exibi-la, remover o primeiro elemento e exibir a lista atualizada.

Exemplo de entrada

Lista original: 13 11 9 7 5 3 1

Exemplo de saída

Lista após remoção do primeiro nó: 11 9 7 5 3 1

- 14 Crie um programa em C++ que delete um nó do meio de uma lista encadeada simples. O programa deve criar uma lista, calcular a posição do meio, remover o nó dessa posição e exibir a lista atualizada. O processo pode ser repetido até que reste apenas um elemento.

Exemplo de entrada

Lista original: 9 7 5 3 1

Exemplo de saída (após primeira remoção)

Lista após remover o meio: 9 7 3 1

Exemplo de saída (após segunda remoção)

Lista após remover o meio: 9 7 1

Exemplo de saída (após terceira remoção)

Lista após remover o meio: 9 1

Exemplo de saída (após quarta remoção)

Lista após remover o meio: 9

- 15 Crie um programa em C++ que delete o último nó de uma lista encadeada simples. O programa deve criar uma lista, exibi-la, remover o último elemento e exibir a lista atualizada. O processo pode ser repetido múltiplas vezes.

Exemplo de entrada

Lista original: 7 5 3 1

Exemplo de saída (após primeira remoção)

Lista após remoção do último nó: 7 5 3

Exemplo de saída (após segunda remoção)

Lista após remoção do último nó: 7 5

- 16 Crie um programa em C++ que implemente um array dinâmico simples com redimensionamento automático. O programa deve criar uma classe que armazena elementos em um array, e quando o array está cheio, cria um novo array com o dobro do tamanho, copia os elementos e libera a memória antiga.

Pseudo-código

```
insert(valor):  
    se tamanho = capacidade:  
        novo_array = novo array[capacidade * 2]  
        copiar elementos para novo_array  
        liberar array antigo  
        array = novo_array  
        capacidade = capacidade * 2  
    array[tamanho] = valor  
    tamanho = tamanho + 1
```

Exemplo de entrada

```
Digite os elementos (0 para parar):  
Elemento 1: 10  
Elemento 2: 20  
Elemento 3: 30  
Elemento 4: 40  
Elemento 5: 50  
Elemento 6: 0
```

Exemplo de saída

```
Capacidade inicial: 4  
Redimensionando: capacidade 4 -> 8  
Array final: 10 20 30 40 50  
Tamanho: 5  
Capacidade: 8
```

- 17 Crie um programa em C++ que implemente uma classe `Vector` com comportamento similar à lista do Python. A classe deve suportar: adicionar elemento no final (`append`), remover e retornar último elemento (`pop`), inserir em posição específica (`insert`), remover primeira ocorrência de valor (`remove`), acesso por índice (`operator[]`), e redimensionamento automático (dobrar quando cheio, reduzir pela metade quando muito vazio).

Pseudo-código

```
append(valor):  
    se tamanho = capacidade: redimensionar()  
    array[tamanho] = valor  
    tamanho = tamanho + 1
```

```
insert(pos, valor):  
    se tamanho = capacidade: redimensionar()  
    para i de tamanho até pos + 1:  
        array[i] = array[i-1]  
    array[pos] = valor  
    tamanho = tamanho + 1
```

```
pop():  
    tamanho = tamanho - 1  
    retornar array[tamanho]
```

```
remove(valor):  
    para i de 0 até tamanho-1:  
        se array[i] = valor:  
            para j de i até tamanho-1:  
                array[j] = array[j+1]  
            tamanho = tamanho - 1  
    retornar
```

Exemplo de interação

```
Vector v;  
v.append(10);  
v.append(20);  
v.append(30);  
v.display();           // [10, 20, 30]  
v.insert(1, 15);       // [10, 15, 20, 30]  
v.remove(20);          // [10, 15, 30]  
int x = v.pop();        // x = 30, v = [10, 15]
```

```
v.display();           // [10, 15]
v[0] = 100;            // [100, 15]
```

Exemplo de saída

Tamanho: 2
Capacidade: 4
Elementos: 100 15

- 18 Crie um programa em C++ que implemente um sistema de Undo/Redo usando listas duplamente ligadas. O sistema deve permitir executar ações (com descrição), desfazer a última ação (undo) e refazer ações desfeitas (redo). Use duas listas: uma para ações executadas e outra para ações desfeitas.

Exemplo de interação

```
> add 10
Ação: Adicionar 10
> add 20
Ação: Adicionar 20
> multiply 2
Ação: Multiplicar por 2
> undo
Desfeito: Multiplicar por 2
> undo
Desfeito: Adicionar 20
> redo
Refeito: Adicionar 20
> display
Ações ativas: Adicionar 10, Adicionar 20
```

Exemplo de saída

Histórico de ações:
[Adicionar 10] <- [Adicionar 20] <- atual
Ações desfeitas: [Multiplicar por 2]

FILAS

- 19 Crie um programa em C++ que implemente a operação de enqueue (enfileirar) em uma fila usando array. O programa deve permitir ao usuário inserir elementos na fila até atingir a capacidade máxima, exibindo mensagens apropriadas.

Exemplo de entrada

```
Capacidade da fila: 5
Digite os elementos (0 para parar):
Elemento: 10
Elemento: 20
Elemento: 30
Elemento: 0
```

Exemplo de saída

```
Fila: [10, 20, 30]
Tamanho: 3
Capacidade: 5
```

- 20 Crie um programa em C++ que implemente a operação de dequeue (desenfileirar) em uma fila usando array. O programa deve permitir remover elementos do início da fila e exibir o elemento removido. Trate o caso de fila vazia.

Exemplo de entrada

```
Fila inicial: [10, 20, 30, 40, 50]
Quantos elementos remover? 3
```

Exemplo de saída

```
Removido: 10
Removido: 20
Removido: 30
Fila atual: [40, 50]
```

- 21 Crie um programa em C++ que implemente a operação de front (consultar início) em uma fila. O programa deve permitir visualizar o elemento do início da fila sem removê-lo, útil para verificar qual elemento será processado a seguir.

Exemplo de entrada

Fila: [15, 25, 35, 45]

Exemplo de saída

Elemento do início: 15

Próximo a ser atendido: 15

Fila permanece: [15, 25, 35, 45]

- 22 Crie um programa em C++ que implemente as operações de size (tamanho) e isEmpty (verificar se vazia) em uma fila. O programa deve fornecer métodos para consultar quantos elementos estão na fila e verificar se ela está vazia antes de operações de remoção.

Exemplo de interação

Fila vazia? Sim

Inserindo: 10, 20, 30

Tamanho: 3

Fila vazia? Não

Removendo todos...

Fila vazia? Sim

Tentando remover de fila vazia: ERRO - Fila vazia!

- 23 Crie um programa em C++ que implemente uma fila completa usando lista encadeada. Ao contrário da implementação com array, a fila deve crescer dinamicamente conforme necessário, sem limite pré-definido de capacidade.

Pseudo-código

```
enqueue(valor):
    novo = criar_novo_no(valor)
    se front = null:
        front = novo
        rear = novo
    senão:
        rear.next = novo
        rear = novo

dequeue():
    valor = front.valor
    front = front.next
    se front = null:
        rear = null
    retornar valor
```

Exemplo de entrada

```
Digite elementos (0 para parar):
Elemento: 5
Elemento: 10
Elemento: 15
Elemento: 20
Elemento: 0
```

Exemplo de saída

```
Fila: 5 -> 10 -> 15 -> 20
Tamanho: 4
Desenfileirando: 5
Fila: 10 -> 15 -> 20
```


- 24 Crie um programa em C++ que simule um sistema de atendimento bancário com múltiplos caixas. Clientes chegam e são atendidos em ordem de chegada (FIFO) pelos caixas disponíveis. O sistema deve calcular métricas como tempo médio de espera na fila e tempo médio de atendimento por caixa.

Cada cliente tem um tempo de atendimento aleatório ou pré-definido. O sistema processa clientes até que todos sejam atendidos, distribuindo-os entre os caixas disponíveis.

Exemplo de entrada

```
Número de caixas: 3
Número de clientes: 8
Tempo de atendimento do cliente 1: 5 min
Tempo de atendimento do cliente 2: 3 min
Tempo de atendimento do cliente 3: 8 min
Tempo de atendimento do cliente 4: 2 min
Tempo de atendimento do cliente 5: 6 min
Tempo de atendimento do cliente 6: 4 min
Tempo de atendimento do cliente 7: 7 min
Tempo de atendimento do cliente 8: 3 min
```

Exemplo de saída

```
Distribuição de atendimento:
Caixa 1: Clientes 1(5min), 4(2min), 7(7min) - Total: 14min
Caixa 2: Clientes 2(3min), 5(6min), 8(3min) - Total: 12min
Caixa 3: Clientes 3(8min), 6(4min) - Total: 12min
```

Estatísticas:

```
Tempo médio de espera na fila: 4.25 min
Tempo médio de atendimento: 4.75 min
Caixa mais ocupado: Caixa 1 (14 min)
Total de clientes atendidos: 8
```

- 25 Crie um programa em C++ que implemente um buffer circular (ring buffer) para comunicação entre processos produtor e consumidor. O buffer tem capacidade fixa e opera com política FIFO circular: quando cheio, o produtor espera; quando vazio, o consumidor espera.

O programa deve simular a produção e consumo de dados, mostrando o estado do buffer a cada operação e como os índices de inserção e remoção circulam pelo array.

Pseudo-código

```
enqueue(valor):  
    se (in + 1) % capacidade = out:  
        buffer cheio - esperar  
    buffer[in] = valor  
    in = (in + 1) % capacidade
```

```
dequeue():  
    se in = out:  
        buffer vazio - esperar  
    valor = buffer[out]  
    out = (out + 1) % capacidade  
    retornar valor
```

Exemplo de simulação

Capacidade do buffer: 4

Produzir: A
Buffer: [A, _, _, _] (in=1, out=0)

Produzir: B
Buffer: [A, B, _, _] (in=2, out=0)

Consumir: A
Buffer: [_, B, _, _] (in=2, out=1)

Produzir: C
Buffer: [_, B, C, _] (in=3, out=1)

Produzir: D
Buffer: [_, B, C, D] (in=0, out=1) // circular!

Produzir: E
Buffer: [E, B, C, D] (in=1, out=1) // sobrescreve? Não,

espera!

Consumir: B

Buffer: [E, _, C, D] (in=1, out=2)

Produzir: E

Buffer: [E, E, C, D] (in=2, out=2)

Exemplo de saída

Total produzido: 5 itens

Total consumido: 2 itens

Itens no buffer: 3

Taxa de ocupação: 75%

- 26 Crie um programa em C++ que implemente uma fila de impressão circular com limite de documentos. O sistema gerencia documentos enviados por múltiplos usuários para uma impressora compartilhada. Quando a fila atinge a capacidade máxima, novos documentos substituem os mais antigos (política de substituição circular).

Cada documento tem: ID, nome do usuário, nome do arquivo e número de páginas. O sistema deve mostrar a ordem de impressão e quais documentos foram substituídos quando a fila estava cheia.

Exemplo de entrada

Capacidade da fila: 5 documentos

Usuário Ana envia: "relatorio.pdf" (10 páginas)

Usuário Bruno envia: "foto.jpg" (1 página)

Usuário Carla envia: "contrato.doc" (5 páginas)

Usuário Daniel envia: "apresentacao.ppt" (15 páginas)

Usuário Elisa envia: "planilha.xlsx" (3 páginas)

Fila cheia! (5/5 documentos)

Usuário Fabio envia: "artigo.pdf" (8 páginas)

Exemplo de saída

Fila de impressão:

1. Bruno - foto.jpg (1 pág)

2. Carla - contrato.doc (5 pág)

3. Daniel - apresentacao.ppt (15 pág)
4. Elisa - planilha.xlsx (3 pág)
5. Fabio - artigo.pdf (8 pág)

Documento substituído: Ana - relatorio.pdf (10 pág)

Total de páginas na fila: 32

Tempo estimado: 6 min 24 seg

PILHAS

- 27 Crie um programa em C++ que implemente uma pilha para gerenciar o histórico de navegação de um navegador web. Cada elemento da pilha deve armazenar a URL da página e o timestamp do acesso.

O programa deve implementar as seguintes operações:

- `push(url, timestamp)`: adiciona uma nova página ao histórico
- `pop()`: remove e retorna a página mais recente (função “Voltar”)
- `top()`: retorna a página atual sem remover
- `isEmpty()`: verifica se o histórico está vazio
- `size()`: retorna a quantidade de páginas no histórico

Exemplo de uso

Histórico: vazio

Visitar: google.com (10:00)

Visitar: github.com (10:05)

Visitar: stackoverflow.com (10:10)

Página atual: stackoverflow.com

Clicar em "Voltar"

Página atual: github.com

Clicar em "Voltar"

Página atual: google.com

Histórico: 1 página

- 28 Crie um programa em C++ que implemente um validador de expressões matemáticas usando pilha. O programa deve verificar se os parênteses (), chaves {} e colchetes [] estão corretamente balanceados em uma expressão.

O programa deve:

- Ler uma expressão contendo os delimitadores
- Usar pilha para verificar o balanceamento
- Retornar verdadeiro se balanceado, falso caso contrário

Exemplo de entrada

Expressão: ([])

Exemplo de saída

Balanceado: true

Exemplo de entrada 2

Expressão: ([])

Exemplo de saída 2

Balanceado: false

- 29** Crie um programa em C++ que implemente um sistema de Undo/Redo para um editor de texto usando duas pilhas. O sistema deve permitir desfazer (undo) e refazer (redo) operações de digitação.

O programa deve implementar:

- `digitar(texto)`: adiciona texto e empilha na pilha de undo, limpa a pilha de redo
- `undo()`: desfaz a última operação (move de undo para redo)
- `redo()`: refaz a operação desfeita (move de redo para undo)
- `getTexto()`: retorna o texto atual do documento

Exemplo de interação

Digitar: "Olá"

Digitar: " Mundo"

Digitar: "!"

Texto: "Olá Mundo!"

Undo

Texto: "Olá Mundo"

Undo

Texto: "Olá"

Digitar: " Pessoal"

Texto: "Olá Pessoal"

Redo (não deve funcionar após nova digitação)

Texto: "Olá Pessoal"

- 30 Crie um programa em C++ que implemente uma calculadora RPN (Reverse Polish Notation) usando pilha. A calculadora deve avaliar expressões em notação pós-fixada.

O programa deve:

- Receber uma lista de tokens (números e operadores)
- Usar pilha para armazenar operandos
- Ao encontrar operador (+, -, *, /), desempilhar dois operandos, aplicar a operação e empilhar o resultado
- Retornar o resultado final

Exemplo de entrada

Expressão: 2 3 + 4 *

Exemplo de saída

Resultado: 20

(Passo 1: empilha 2, 3)

(Passo 2: encontra +, desempilha 3 e 2, calcula 2+3=5, empilha 5)

(Passo 3: empilha 4)

(Passo 4: encontra *, desempilha 4 e 5, calcula 5*4=20)

- 31 Crie um programa em C++ que simule a pilha de chamadas de funções (call stack) de um programa. Cada elemento da pilha deve armazenar o nome da função e o endereço de retorno.

O programa deve implementar:

- `pushFrame(funcao, retorno)`: empilha uma nova chamada de função
- `popFrame()`: desempilha ao retornar da função
- `topFrame()`: retorna a função atual em execução
- `displayStack()`: exibe a pilha de chamadas atual

Exemplo de simulação

main() chama A()
Pilha: [main]

A() chama B()
Pilha: [main, A]

B() chama C()

Pilha: [main, A, B]

C() retorna

Pilha: [main, A, B]

B() retorna

Pilha: [main, A]

A() chama D()

Pilha: [main, A, D]

- 32 Crie um programa em C++ que converta números decimais para outras bases (binário, hexadecimal, octal) usando pilha. O programa deve armazenar os restos das divisões sucessivas na pilha e depois desempilhar para formar o resultado.

O programa deve:

- Receber um número decimal e a base de destino (2, 8 ou 16)
- Usar pilha para armazenar os restos da divisão
- Desempilhar todos os restos para formar a representação na nova base
- Suportar bases até 16 (dígitos 0-9, A-F)

Exemplo de entrada

Número: 45

Base: 2

Exemplo de saída

45 em base 2: 101101

(Divisões: 45/2=22 r1, 22/2=11 r0, 11/2=5 r1, 5/2=2 r1, 2/2=1 r0, 1/2=0 r1)

- 33 Crie um programa em C++ que implemente um algoritmo de busca em profundidade (DFS) para encontrar um caminho em um labirinto usando pilha. O algoritmo deve empilhar as posições visitadas e desempilhar ao retroceder.

O programa deve:

- Representar o labirinto como uma matriz (0 = livre, 1 = parede)

- Usar pilha para armazenar as posições (x, y) do caminho
- Marcar posições visitadas
- Desempilhar quando encontra beco sem saída
- Retornar o caminho encontrado da entrada até a saída

Exemplo de labirinto

```
Entrada → E . . # .
        . # . # .
        . . . . S ← Saída
        # # # # .
```

Caminho encontrado:

```
(0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4)
```

- 34 Crie um programa em C++ que implemente um sistema de desfazer (undo) para um aplicativo de desenho. Cada traço (linha) desenhado deve ser empilhado para permitir desfazer as operações.

O programa deve implementar:

- Estrutura **Traço** com coordenadas (x1, y1) e (x2, y2)
- `adicionarTraço(x1, y1, x2, y2)`: adiciona novo traço à pilha
- `undo()`: remove o último traço adicionado
- `listarTracos()`: exibe todos os traços atuais
- `count()`: retorna quantidade de traços

Exemplo de interação

```
Desenhar: (10,10) - (20,20)
Desenhar: (30,10) - (40,20)
Desenhar: (50,10) - (60,20)
Desenhar: (70,10) - (80,20)
Desenhar: (90,10) - (100,20)
```

Total de traços: 5

```
Undo
Undo
Undo
```

Total de traços: 2

```
Desenhar: (110,10) - (120,20)
```

Desenhar: (130,10) - (140,20)

Total de traços: 4

- 35 Crie um programa em C++ que implemente o algoritmo Shunting-Yard para converter expressões matemáticas da notação infixa para pós-fixa (notação polonesa reversa) usando pilha.

O programa deve:

- Receber uma expressão infixa (ex: $3 + 4 * 2 / (1 - 5)$)
- Usar pilha para armazenar operadores (+, -, *, /, (,))
- Aplicar as regras de precedência e associatividade
- Retornar a expressão em notação pós-fixa

Exemplo de entrada

Expressão infixa: $3 + 4 * 2 / (1 - 5)$

Exemplo de saída

Expressão pós-fixa: $3 4 2 * 1 5 - / +$

Passos:

- 3 → saída
- + → pilha
- 4 → saída
- * → pilha (maior precedência que +)
- 2 → saída
- / → desempilha *, empilha /
- (→ empilha (
- 1 → saída
- - → empilha -
- 5 → saída
-) → desempilha até (, envia - para saída
- Fim: desempilha tudo

- 36 Crie um programa em C++ que verifique se uma palavra ou frase é um palíndromo usando pilha. Um palíndromo é uma sequência que pode ser lida da mesma forma de trás para frente.

O programa deve:

- Receber uma string (ignorar espaços e diferença entre maiúsculas/minúsculas)
- Empilhar a primeira metade da string
- Comparar a segunda metade desempilhando caractere por caractere
- Retornar verdadeiro se for palíndromo, falso caso contrário

Exemplo de entrada

Palavra: radar

Exemplo de saída

Palíndromo: true
(Empilha: r, a, d)
(Compara: d==d, a==a, r==r)

Exemplo de entrada 2

Palavra: hello

Exemplo de saída 2

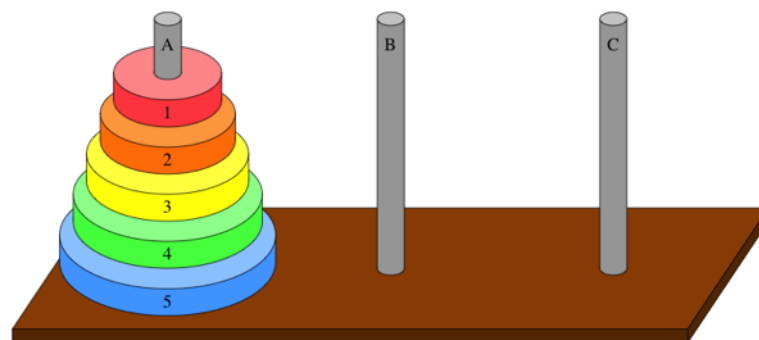
Palíndromo: false
(Empilha: h, e, l)
(Compara: l!=o)

RECURSÃO

- 37 Crie um programa em C++ que resolva o problema das Torres de Hanoi. O objetivo é mover N discos de tamanhos diferentes do pino A (origem) para o pino C (destino), usando o pino B como auxiliar.

Configuração inicial:

- Pino A: N discos empilhados, do maior (base) ao menor (topo)
- Pino B: vazio
- Pino C: vazio



Regras:

1. Mover apenas um disco por vez
2. Nunca colocar um disco maior sobre um menor
3. Usar o pino B como auxiliar quando necessário

Estratégia recursiva: Para mover n discos de Origem para Destino usando Auxiliar:

1. Mover n-1 discos de Origem para Auxiliar (usando Destino)
2. Mover o disco n (maior) de Origem para Destino
3. Mover n-1 discos de Auxiliar para Destino (usando Origem)

Pseudo-código

```
hanoi(n, origem, destino, auxiliar):  
    se n == 1:  
        imprime "Disco 1: origem → destino"  
    senão:  
        hanoi(n-1, origem, auxiliar, destino)  
        imprime "Disco n: origem → destino"  
        hanoi(n-1, auxiliar, destino, origem)
```

O programa deve:

- Receber o número de discos (N)

- Implementar a função recursiva `hanoi(n, origem, destino, auxiliar)`
- Exibir cada movimento no formato “Disco X: A → C”
- Mostrar o número total de movimentos ($2^N - 1$)

Exemplo de entrada

Número de discos: 3

Exemplo de saída

Resolvendo Torres de Hanoi com 3 discos:

Movimentos:

Disco 1: A → C

Disco 2: A → B

Disco 1: C → B

Disco 3: A → C

Disco 1: B → A

Disco 2: B → C

Disco 1: A → C

Total de movimentos: 7

- 38** Crie um programa em C++ que gere todas as permutações de uma string usando recursão e backtracking. O programa deve trocar caracteres e gerar todas as combinações possíveis.

O programa deve:

- Receber uma string
- Implementar a função recursiva `permutar(str, inicio, fim)`
- Trocar caracteres e fazer chamadas recursivas para gerar permutações
- Exibir todas as permutações encontradas

Exemplo de entrada

String: ABC

Exemplo de saída

Permutações:

ABC

ACB

BAC
BCA
CAB
CBA

Total: 6 permutações

- 39** Crie um programa em C++ que implemente duas funções recursivas para manipulação de números inteiros: soma dos dígitos e inversão do número.

O programa deve implementar:

- `somaDigitos(n)`: retorna a soma de todos os dígitos do número
- `inverterNumero(n)`: retorna o número com os dígitos na ordem inversa

Exemplo de entrada

Número: 1234

Exemplo de saída

Soma dos dígitos: 10
(1 + 2 + 3 + 4 = 10)

Número invertido: 4321
(1234 → 4321)

Exemplo de entrada 2

Número: 987

Exemplo de saída 2

Soma dos dígitos: 24
(9 + 8 + 7 = 24)

Número invertido: 789
(987 → 789)

ORDENAÇÃO

- 40 Crie um programa em C++ que implemente o Bubble Sort para ordenar um array de inteiros em ordem crescente.

Pseudo-código

```
para i de 0 até n-2:  
    para j de 0 até n-i-2:  
        se A[j] > A[j+1]:  
            trocar(A[j], A[j+1])
```

Exemplo de entrada

```
Digite o tamanho do array: 6  
Digite os elementos:  
Elemento 1: 64  
Elemento 2: 34  
Elemento 3: 25  
Elemento 4: 12  
Elemento 5: 22  
Elemento 6: 11
```

Exemplo de saída

```
Array original: 64 34 25 12 22 11  
Array ordenado: 11 12 22 25 34 64  
Total de comparações: 15  
Total de trocas: 9
```

- 41 Crie um programa em C++ que implemente o Selection Sort para ordenar um array de inteiros.

Pseudo-código

```
para i de 0 até n-1:
    min = i
    para j de i+1 até n-1:
        se A[j] < A[min]:
            min = j
    se min ≠ i:
        trocar(A[i], A[min])
```

Exemplo de entrada

```
Digite o tamanho do array: 5
Digite os elementos:
Elemento 1: 29
Elemento 2: 10
Elemento 3: 14
Elemento 4: 37
Elemento 5: 13
```

Exemplo de saída

```
Array original: 29 10 14 37 13
Passo 1: Encontrou menor 10, trocou com posição 0: 10 29 14 37 13
Passo 2: Encontrou menor 13, trocou com posição 1: 10 13 14 37 29
Passo 3: Encontrou menor 14, trocou com posição 2: 10 13 14 37 29
Passo 4: Encontrou menor 29, trocou com posição 3: 10 13 14 29 37
Array ordenado: 10 13 14 29 37
```


- 42 Crie um programa em C++ que implemente o Insertion Sort para ordenar um array de inteiros.

Pseudo-código

```
para i de 1 até n-1:
    chave = A[i]
    j = i - 1
    enquanto j ≥ 0 e A[j] > chave:
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = chave
```

Exemplo de entrada

```
Digite o tamanho do array: 6
Digite os elementos:
Elemento 1: 12
Elemento 2: 11
Elemento 3: 13
Elemento 4: 5
Elemento 5: 6
Elemento 6: 2
```

Exemplo de saída

```
Array original: 12 11 13 5 6 2
Inserindo 11: 11 12 13 5 6 2
Inserindo 13: 11 12 13 5 6 2
Inserindo 5: 5 11 12 13 6 2
Inserindo 6: 5 6 11 12 13 2
Inserindo 2: 2 5 6 11 12 13
Array ordenado: 2 5 6 11 12 13
```

- 43 Crie um programa em C++ que implemente o Merge Sort.

Pseudo-código

```
mergeSort(A, esq, dir):  
    se esq < dir:  
        meio = (esq + dir) / 2  
        mergeSort(A, esq, meio)  
        mergeSort(A, meio+1, dir)  
        merge(A, esq, meio, dir)  
  
merge(A, esq, meio, dir):  
    // Combinar dois subarrays ordenados
```

Exemplo de entrada

```
Digite o tamanho do array: 8  
Digite os elementos:  
Elemento 1: 38  
Elemento 2: 27  
Elemento 3: 43  
Elemento 4: 3  
Elemento 5: 9  
Elemento 6: 82  
Elemento 7: 10  
Elemento 8: 50
```

Exemplo de saída

```
Array original: 38 27 43 3 9 82 10 50  
Dividindo: [38 27 43 3] [9 82 10 50]  
Dividindo: [38 27] [43 3]  
Dividindo: [38] [27]  
Mesclando [27 38]  
Dividindo: [43] [3]  
Mesclando [3 43]  
Mesclando [3 27 38 43]  
Dividindo: [9 82] [10 50]  
Dividindo: [9] [82]  
Mesclando [9 82]  
Dividindo: [10] [50]  
Mesclando [10 50]  
Mesclando [9 10 50 82]  
Mesclando [3 9 10 27 38 43 50 82]  
Array ordenado: 3 9 10 27 38 43 50 82
```

- 44 Crie um programa em C++ que implemente o Quick Sort.

Pseudo-código

```
quickSort(A, esq, dir):  
    se esq < dir:  
        p = particionar(A, esq, dir)  
        quickSort(A, esq, p-1)  
        quickSort(A, p+1, dir)
```

```
particionar(A, esq, dir):  
    pivô = A[(esq + dir) / 2]  
    // Particionar ao redor do pivô
```

Exemplo de entrada

```
Digite o tamanho do array: 7  
Digite os elementos:  
Elemento 1: 25  
Elemento 2: 13  
Elemento 3: 6  
Elemento 4: 42  
Elemento 5: 17  
Elemento 6: 8  
Elemento 7: 30
```

Exemplo de saída

```
Array original: 25 13 6 42 17 8 30  
Particionando [25 13 6 42 17 8 30], pivô: 17  
Esquerda: [13 6 8] | Direita: [25 42 30]  
Particionando [13 6 8], pivô: 6  
Esquerda: [] | Direita: [13 8]  
Particionando [13 8], pivô: 8  
Esquerda: [] | Direita: [13]  
Ordenado: [6 8 13]  
Particionando [25 42 30], pivô: 42  
Esquerda: [25 30] | Direita: []  
Particionando [25 30], pivô: 30  
Esquerda: [25] | Direita: []  
Ordenado: [25 30 42]  
Array ordenado: 6 8 13 17 25 30 42
```

EXERCÍCIOS DE ENADE

COMPUTAÇÃO 2011 ([GABARITO](#))

- 20 P2** Considere que G é um grafo qualquer e que V e E são os conjuntos de vértices e de arestas de G , respectivamente. Considere também que grau (v) é o grau de um vértice v pertencente ao conjunto V . Nesse contexto, analise as seguintes asserções.

Em G , a quantidade de vértices com grau ímpar é ímpar.

PORQUE

$$\sum_{v \in V} \text{grau}(v) = 2|E|$$

Para G , vale a identidade dada pela expressão

Acerca dessas asserções, assinale a opção correta.

- A** As duas asserções são proposições verdadeiras, e a segunda é uma justificativa correta da primeira.
- B** As duas asserções são proposições verdadeiras, mas a segunda não é uma justificativa correta da primeira.
- C** A primeira asserção é uma proposição verdadeira, e a segunda uma proposição falsa.
- D** A primeira asserção é uma proposição falsa, e a segunda uma proposição verdadeira.
- E** Tanto a primeira quanto a segunda asserções são proposições falsas.

- 21 P1** No desenvolvimento de um software que analisa bases de DNA, representadas pelas letras A, C, G, T, utilizou-se as estruturas de dados: pilha e fila. Considere que, se uma sequência representa uma pilha, o topo é o elemento mais à esquerda; e se uma sequência representa uma fila, a sua frente é o elemento mais à esquerda.

Analise o seguinte cenário: “a sequência inicial ficou armazenada na primeira estrutura de dados na seguinte ordem: (A,G,T,C,A,G,T,T). Cada elemento foi retirado da primeira estrutura de dados e inserido na segunda estrutura de dados, e a sequência ficou armazenada na seguinte ordem: (T,T,G,A,C,T,G,A). Finalmente, cada elemento foi retirado da segunda estrutura de dados e inserido na terceira estrutura de dados e a sequência ficou armazenada na seguinte ordem: (T,T,G,A,C,T,G,A)”.

Qual a única sequência de estruturas de dados apresentadas a seguir pode ter sido usada no cenário descrito acima?

- A** Fila - Pilha - Fila.
- B** Fila - Fila - Pilha.
- C** Fila - Pilha - Pilha.
- D** Pilha - Fila - Pilha.
- E** Pilha - Pilha - Pilha.

- 23 P2** As filas de prioridades (heaps) são estruturas de dados importantes no projeto de algoritmos. Em especial, heaps podem ser utilizados na recuperação de informação em grandes bases de dados constituídos por textos. Basicamente, para se exibir o resultado de uma consulta, os documentos recuperados são ordenados de acordo com a relevância presumida para o usuário. Uma consulta pode recuperar milhões de documentos que certamente não serão todos examinados. Na verdade, o usuário examina os primeiros m documentos dos n recuperados, em que m é da ordem de algumas dezenas.

Considerando as características dos heaps e sua aplicação no problema descrito acima, avalie as seguintes afirmações.

- I. Uma vez que o heap é implementado como uma árvore binária de pesquisa essencialmente completa, o custo computacional para sua construção é $O(n \log n)$.
- II. A implementação de heaps utilizando-se vetores é eficiente em tempo de execução e em espaço de armazenamento, pois o pai de um elemento armazenado na posição i se encontra armazenado na posição $2i+1$.
- III. O custo computacional para se recuperar de forma ordenada os m documentos mais relevantes armazenados em um heap de tamanho n é $O(m \log n)$.
- IV. Determinar o documento com maior valor de relevância armazenado em um heap tem custo computacional $O(1)$.

Está correto apenas o que se afirma em

- A** I e II.
- B** II e III.
- C** III e IV.
- D** I, II e IV.
- E** I, III e IV.

- 27 P1** Algoritmos criados para resolver um mesmo problema podem diferir de forma drástica quanto a sua eficiência. Para evitar este fato, são utilizadas técnicas algorítmicas, isto é, conjunto de técnicas que compreendem os métodos de codificação de algoritmos de forma a salientar sua complexidade, levando-se em conta a forma pela qual determinado algoritmo chega à solução desejada.

Considerando os diferentes paradigmas e técnicas de projeto de algoritmos, analise as afirmações abaixo.

- I. A técnica de tentativa e erro (backtracking) efetua uma escolha ótima local, na esperança de obter uma solução ótima global.
- II. A técnica de divisão e conquista pode ser dividida em três etapas: dividir a instância do problema em duas ou mais instâncias menores; resolver as instâncias menores recursivamente; obter a solução para as instâncias originais (maiores) por meio da combinação dessas soluções.
- III. A técnica de programação dinâmica decompõe o processo em um número finito de subtarefas parciais que devem ser exploradas exaustivamente.
- IV. O uso de heurísticas (ou algoritmos aproximados) é caracterizado pela ação de um procedimento chamar a si próprio, direta ou indiretamente.

É correto apenas o que se afirma em

- A** I.
- B** II.
- C** I e IV.
- D** II e III.
- E** III e IV.

- 29 P2** Suponha que se queira pesquisar a chave 287 em uma árvore binária de pesquisa com chaves entre 1 e 1 000. Durante uma pesquisa como essa, uma sequência de chaves é examinada. Cada sequência abaixo é uma suposta sequência de chaves examinadas em uma busca da chave 287.

I. 7, 342, 199, 201, 310, 258, 287

II. 110, 132, 133, 156, 289, 288, 287

III. 252, 266, 271, 294, 295, 289, 287

IV. 715, 112, 530, 249, 406, 234, 287

É válido apenas o que se apresenta em

A I.

B III.

C I e II.

D II e IV.

E III e IV.

- 2 P1** Os números de Fibonacci correspondem à uma sequência infinita na qual os dois primeiros termos são 0 e 1. Cada termo da sequência, à exceção dos dois primeiros, é igual à soma dos dois anteriores, conforme a relação de recorrência abaixo.

$$f_n = f_{n-1} + f_{n-2}$$

Desenvolva dois algoritmos, um iterativo e outro recursivo, que, dado um número natural $n > 0$, retorna o n -ésimo termo da sequência de Fibonacci. Apresente as vantagens e desvantagens de cada algoritmo.

- ③ **P2** Listas ordenadas implementadas com vetores são estruturas de dados adequadas para a busca binária, mas possuem o inconveniente de exigirem custo computacional de ordem linear para a inserção de novos elementos. Se as operações de inserção ou remoção de elementos forem frequentes, uma alternativa é transformar a lista em uma árvore binária de pesquisa balanceada, que permitirá a execução dessas operações com custo logarítmico.

Considerando essas informações, escreva um algoritmo recursivo que construa uma árvore binária de pesquisa completa, implementada por estruturas auto-referenciadas ou apontadores, a partir de um vetor ordenado, v , de n inteiros, em que $n = 2^m - 1$, $m > 0$. O algoritmo deve construir a árvore em tempo linear, sem precisar fazer qualquer comparação entre os elementos do vetor, uma vez que este já está ordenado. Para isso,

- a) descreva a estrutura de dados utilizada para a implementação da árvore (valor = 2,0 pontos)
- b) escreva o algoritmo para a construção da árvore. A chamada principal à função recursiva deve passar, como parâmetros, o vetor, índice do primeiro e último elementos, retornando a referência ou apontador para a raiz da árvore criada (valor: 8,0 pontos).

Observação: Qualquer notação em português estruturado, de forma imperativa ou orientada a objetos deve ser considerada, assim como em uma linguagem de alto nível, como o Pascal, C e Java.

BACHARELADO EM COMPUTAÇÃO 2014 (GABARITO)

- 3 P2 O jogo Sudoku consiste em uma matriz 9x9 dividida em 9 sub-matrizes 3x3, como mostrado na figura a seguir.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

A matriz está parcialmente preenchida com números de 1 a 9, e o objetivo do jogo é completar a matriz, de forma que cada linha, coluna e sub-matriz contenham todos os números de 1 a 9.

A partir dessas informações, escreva um algoritmo recursivo baseado em retrocesso (backtracking) para resolver o jogo. A matriz foi transformada em um vetor V de 81 posições, contendo zeros nas posições que faltam para serem preenchidas. Considere que existem duas funções implementadas. A primeira função, $NaoHaViolacao(x,i,V)$, retorna verdadeiro se a inserção do número x na posição i do vetor V não causa violação das restrições do jogo (número repetido em linha, coluna ou sub-matriz). A segunda função, $Imprime(V)$, realiza a impressão do vetor V . Considere, ainda, que o algoritmo deve imprimir o vetor V com a solução encontrada, se esta existir. (valor: 10,0 pontos)

Observação: Qualquer notação em português estruturado, de forma imperativa ou orientada a objetos pode ser utilizada, assim como em uma linguagem de alto nível, como Pascal, C ou Java.

5 P1 As técnicas de projeto de algoritmos são essenciais para que os desenvolvedores possam implementar software de qualidade. Essas técnicas descrevem os princípios que devem ser adotados para se projetar soluções algorítmicas para um dado problema. Entre as principais técnicas, destacam-se os projetos de algoritmos por tentativa e erro, divisão e conquista, programação dinâmica e algoritmos gulosos. Nesse contexto, faça o que se pede nos itens a seguir.

a Descreva o que caracteriza o projeto de algoritmos por divisão e conquista. (valor: 6,0 pontos)

b Apresente uma situação de uso da técnica de projeto de algoritmos por divisão e conquista. (valor: 4,0 pontos)

12 P1 Analise o custo computacional dos algoritmos a seguir, que calculam o valor de um polinômio de grau n , da forma: $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, onde os coeficientes são números de ponto flutuante armazenados no vetor $a[0..n]$, e o valor de n é maior que zero. Todos os coeficientes podem assumir qualquer valor, exceto o coeficiente a_n que é diferente de zero.

Algoritmo 1:

```
soma = a[0]
Repita para i = 1 até n
  Se a[i] ≠ 0.0 então
    potência = x
    Repita para J = 2 até i
      potência = potência * x
    Fim repita
    soma = soma + a[i] * potência
  Fim se
Fim repita
Imprima(soma)
```

Algoritmo 2:

```
soma = a[n]
Repita para i = n-1 até 0 passo -1
  soma = soma * x + a[i]
```

```
Fim repita  
Imprima(soma)
```

Com base nos algoritmos 1 e 2, avalie as asserções a seguir e a relação proposta entre elas.

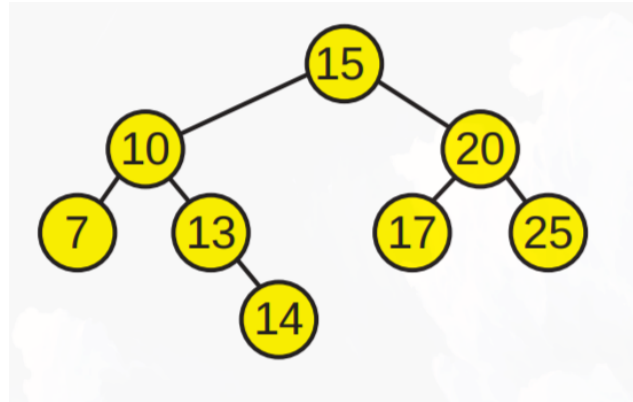
I. Os algoritmos possuem a mesma complexidade assintótica.

PORQUE

II. Para o melhor caso, ambos os algoritmos possuem complexidade $O(n)$.

A respeito dessas asserções, assinale a opção correta.

- ☐ A As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
 - ☐ B As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
 - ☐ C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
 - ☐ D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
 - ☐ E As asserções I e II são proposições falsas.
- 13 P2** A figura a seguir apresenta uma árvore binária de pesquisa, que mantém a seguinte propriedade fundamental: o valor associado à raiz é sempre menor do que o valor de todos os nós da subárvore à direita e sempre maior do que o valor de todos os nós da subárvore à esquerda.



- I. A árvore possui a vantagem de realizar a busca de elementos de forma eficiente, como a busca binária em um vetor.
- II. A árvore está desbalanceada, pois a subárvore da esquerda possui um número de nós maior do que a subárvore da direita.
- III. Quando a árvore é percorrida utilizando o método de caminamento pós-ordem, os valores são encontrados em ordem decrescente.
- IV. O número de comparações realizadas em função do número n de elementos na árvore em uma busca binária realizada com sucesso é $O(\log n)$.

É correto apenas o que se afirma em:

- ☒ A I e III.
- ☐ B I e IV.
- ☐ C II e III.
- ☐ D I, II e IV.
- ☐ E II, III e IV.

- ⑩ **P1** Uma pilha é uma estrutura de dados que armazena uma coleção de itens de dados relacionados e que garante o seguinte funcionamento: o último elemento a ser inserido é o primeiro a ser removido. É comum na literatura utilizar os nomes **push** e **pop** para as operações de inserção e remoção de um elemento em uma pilha, respectivamente. O seguinte trecho de código em linguagem C define uma estrutura de dados pilha utilizando um vetor de inteiros, bem como algumas funções para sua manipulação.

```
#include <stdlib.h>
#include <stdio.h>
typedef struct {
    int elementos[100];
    int topo;
} pilha;

pilha * cria_pilha() {
    pilha * p = malloc(sizeof(pilha));
    p->topo = -1;
    return p;
}

void push(pilha *p, int elemento) {
    if (p->topo >= 99)
        return;
    p->elementos[++p->topo] = elemento;
}

int pop(pilha *p) {
    int a = p->elementos[p->topo];
    p->topo--;
    return a;
}
```

O programa a seguir utiliza uma pilha.

```
int main() {
    pilha * p = cria_pilha();
    push(p, 2);
    push(p, 3);
    push(p, 4);
    pop(p);
    push(p, 2);
    int a = pop() + pop(p);
    push(p, a);
}
```

```
    a += pop(p);  
    printf("%d", a);  
    return 0;  
}
```

A esse respeito, avalie as afirmações a seguir.

- I. A complexidade computacional de ambas funções push e pop é $O(1)$.
- II. O valor exibido pelo programa seria o mesmo caso a instrução `a += pop(p);` fosse trocada por `a += a;`
- III. Em relação ao vazamento de memória (memory leak), é opcional chamar a função `free(p)`, pois o vetor usado pela pilha é alocado estaticamente.

É correto o que se afirma em

- ☐ A I, apenas.
- ☐ B II, apenas.
- ☐ C I e II, apenas.
- ☐ D II e III, apenas.
- ☐ E I, II e III.

23 P1 Qual o valor de retorno da função a seguir, caso $n = 27$?

```
int recursao (int n) {  
    if (n <= 10) {  
        return n * 2;  
    }  
    else {  
        return recursao(recursao(n/3));  
    }  
}
```

A 8.

B 9.

C 12.

D 16.

E 18.

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO 2017 ([GABARITO](#))

- ③ P1 Listas lineares armazenam uma coleção de elementos. A seguir, é apresentada a declaração de uma lista simplesmente encadeada.

```
struct ListaEncadeada {  
    int dado;  
    struct ListaEncadeada *proximo;  
};
```

Para imprimir os seus elementos da cauda para a cabeça (do final para o início) de forma eficiente, um algoritmo pode ser escrito da seguinte forma:

```
void mostrar(struct ListaEncadeada *lista) {  
    if (lista != NULL) {  
        mostrar(lista->proximo);  
        printf("%d ", lista->dado);  
    }  
}
```

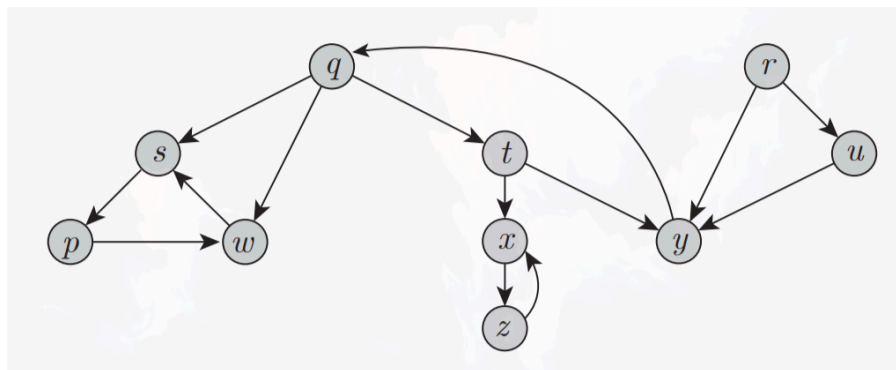
Com base no algoritmo apresentado, faça o que se pede nos itens a seguir.

- a) Apresente a classe de complexidade do algoritmo, usando a notação Θ . (valor: 3,0 pontos)
- b) Considerando que já existe implementada uma estrutura de dados do tipo pilha de inteiros — com as operações de empilhar, desempilhar e verificar pilha vazia — reescreva o algoritmo de forma não recursiva, mantendo a mesma complexidade. Seu algoritmo pode ser escrito em português estruturado ou em alguma linguagem de programação, como C, Java ou Pascal. (valor: 7,0 pontos)

- 5 **P2** A busca primeiro em profundidade é um algoritmo de exploração sistemática em grafos, em que as arestas são exploradas a partir do vértice v mais recentemente descoberto que ainda tem arestas inexploradas saindo dele. Quando todas as arestas de v são exploradas, a busca regressa para explorar as arestas que deixam o vértice a partir do qual v foi descoberto. Esse processo continua até que todos os vértices acessíveis a partir do vértice inicial sejam explorados.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to algorithms*. 3. ed. Cambridge, Massachusetts: The MIT Press, 2009 (adaptado).

Considere o grafo a seguir.



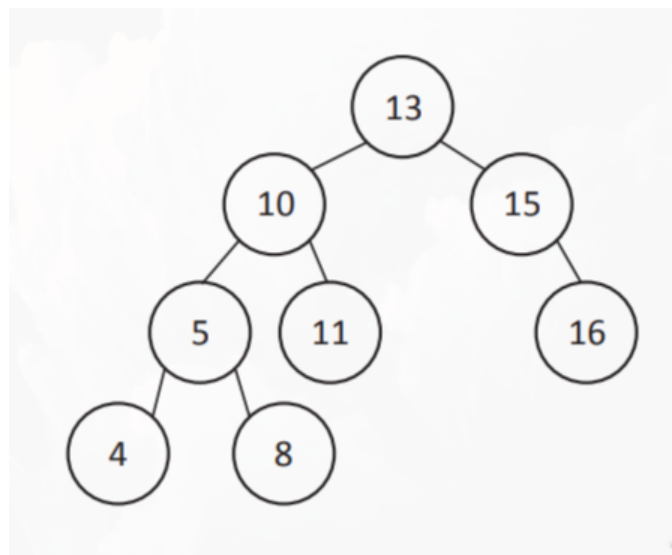
Com base nas informações apresentadas, faça o que se pede nos itens a seguir.

- a Mostre a sequência de vértices descobertos no grafo durante a execução de uma busca em profundidade com controle de estados repetidos. Para isso, utilize o vértice r como inicial. No caso de um vértice explorado ter mais de um vértice adjacente, utilize a ordem alfabética crescente como critério para priorizar a exploração. (valor: 7,0 pontos)
- b Faça uma representação da matriz de adjacências desse grafo, podendo os zeros ser omitidos nessa matriz. (valor: 3,0 pontos)

- 9 **P2** Uma árvore AVL é um tipo de árvore binária balanceada na qual a diferença entre as alturas de suas subárvores da esquerda e da direita não pode ser maior do que 1 para qualquer nó. Após a inserção de um nó em uma AVL, a raiz da subárvore de nível mais baixo no qual o novo nó foi inserido é marcada. Se a altura de seus filhos diferir em mais de uma unidade, é realizada uma rotação simples ou uma rotação dupla para igualar suas alturas.

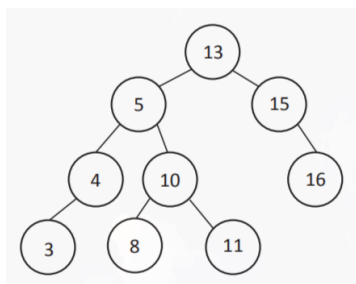
LAFORE, R. *Data structures & algorithms in Java*. Indianápolis: Sams Publishing, 2003 (adaptado).

A seguir, é apresentado um exemplo de árvore AVL.

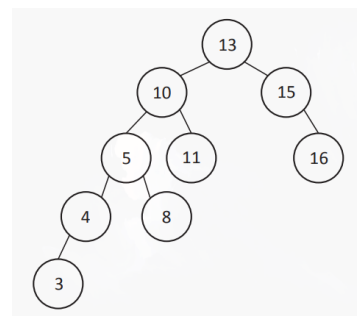


Pelo exposto no texto acima, após a inserção de um nó com valor 3 na árvore AVL exemplificada, é correto afirmar que ela ficará com a seguinte configuração:

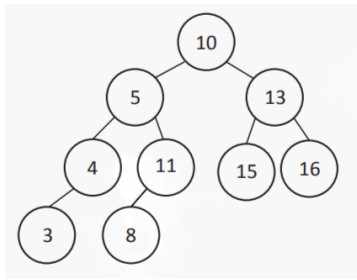
A



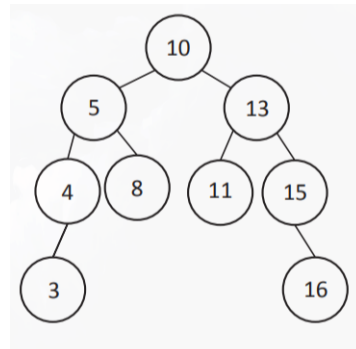
B



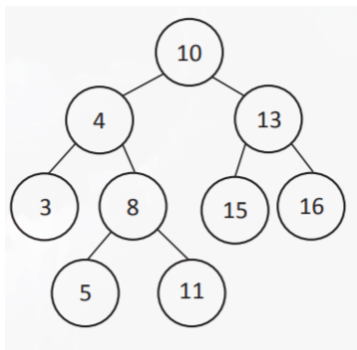
C



D



E



- 18 P1** O algoritmo a seguir recebe um vetor *V* de números inteiros e rearranja esse vetor de tal forma que seus elementos, ao final, estejam ordenados de forma crescente.

```
01 void ordena(int *v, int n)
02 {
03     int i, j, chave;
04     for(i = 1; i < n; i++)
05     {
06         chave = v[i];
07         j = i - 1;
08         while(j >= 0 && v[j] < chave)
09         {
10             v[j-1] = v[j];
11             j = j - 1;
12         }
13         v[j+1] = chave;
14     }
15 }
```

Considerando que nesse algoritmo há erros de lógica que devem ser corrigidos para que os elementos sejam ordenados de forma crescente, assinale a opção correta no que se refere às correções adequadas.

- A** A linha 04 deve ser corrigida da seguinte forma: `for(i = 1; i < n - 1; i++)` e a linha 13, do seguinte modo: `v[j - 1] = chave;`
- B** A linha 04 deve ser corrigida da seguinte forma: `for(i = 1; i < n - 1; i++)` e a linha 07, do seguinte modo: `j = i + 1;`
- C** A linha 07 deve ser corrigida da seguinte forma: `j = i + 1` e a linha 08, do seguinte modo: `while(j >= 0 && v[j] > chave)`
- D** A linha 08 deve ser corrigida da seguinte forma: `while(j >= 0 && v[j] >= chave)` e a linha 10, do seguinte modo: `v[j + 1] = v[j];`
- E** A linha 10 deve ser corrigida da seguinte forma: `v[j + 1] = v[j];` e a linha 13, do seguinte modo: `v[j - 1] = chave;`

- 22 P2** Um país utiliza moedas de 1, 5, 10, 25 e 50 centavos. Um programador desenvolveu o método a seguir, que implementa a estratégia gulosa para o problema do troco mínimo. Esse método recebe como parâmetro um valor inteiro, em centavos, e retorna um *array* no qual cada posição indica a quantidade de moedas de cada valor.

```
public static int[] troco(int valor){
    int[] moedas = new int[5];

    moedas[4] = valor / 50;
    valor = valor % 50;
    moedas[3] = valor / 25;
    valor = valor % 25;
    moedas[2] = valor / 10;
    valor = valor % 10;
    moedas[1] = valor / 5;
    valor = valor % 5;
    moedas[0] = valor;
    return(moedas);
}
```

Considerando o método apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. O método guloso encontra o menor número de moedas para o valor de entrada, considerando as moedas do país.

PORQUE

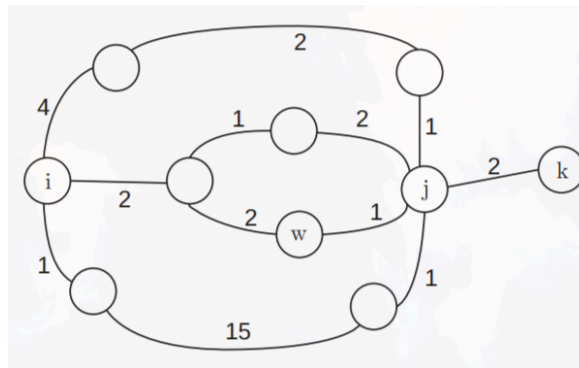
II. Métodos gulosos sempre encontram a solução global ótima.

A respeito dessas asserções, assinale a opção correta.

- A** As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B** As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C** A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D** A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

(E) As asserções I e II são proposições falsas.

- (24) **P2** A figura a seguir exhibe um grafo que representa um mapa rodoviário, no qual os vértices representam cidades e as arestas representam vias. Os pesos indicam o tempo atual de deslocamento entre duas cidades.



Considerando que os tempos de ida e volta são iguais para qualquer via, avalie as afirmações a seguir acerca desse grafo.

- I. Dado o vértice de origem **i**, o algoritmo Dijkstra encontra o menor tempo de deslocamento entre a cidade **i** e todas as demais cidades do grafo.
- II. Uma árvore geradora de custo mínimo gerada pelo algoritmo de Kruskal contém um caminho de custo mínimo cujo origem é **i** e cujo destino é **k**.
- III. Se um caminho de custo mínimo entre os vértices **i** e **k** contém o vértice **w**, então o subcaminho de origem **w** e destino **k** deve também ser mínimo.

É correto o que se afirma em

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.

(E) I, II e III.

- (25) P1** A sequência de Fibonacci é uma sequência de números inteiros que começa em 1, a que se segue 1, e na qual cada elemento subsequente é a soma dos dois elementos anteriores. A função `fib` a seguir calcula o n -ésimo elemento da sequência de Fibonacci:

```
unsigned int fib (unsigned int n)
{
    if (n < 2)
        return 1;
    return fib(n - 2) + fib(n - 1);
}
```

Considerando a implementação acima, avalie as afirmações a seguir.

- I. A complexidade de tempo da função `fib` é exponencial no valor de n .
- II. A complexidade de espaço da função `fib` é exponencial no valor de n .
- III. É possível implementar uma versão iterativa da função `fib` com complexidade de tempo linear no valor de n e complexidade de espaço constante.

É correto o que se afirma em

- (A)** I, apenas.
- (B)** II, apenas.
- (C)** I e III, apenas.
- (D)** II e III, apenas.
- (E)** I, II e III.

- 33 P1 Considere a função recursiva F a seguir, que em sua execução chama a função G:

```
1 void F(int n) {  
2     if(n > 0) {  
3         for(int i = 0; i < n; i++) {  
4             G(i);  
5         }  
6         F(n/2);  
7     }  
8 }
```

Com base nos conceitos de teoria da complexidade, avalie as afirmações a seguir.

I. A equação de recorrência que define a complexidade da função F é a mesma do algoritmo clássico de ordenação *mergesort*. II. O número de chamadas recursivas da função F é $\Theta(\log n)$. III. O número de vezes que a função G da linha 4 é chamada é $O(n \log n)$.

É correto o que se afirma em

- A I, apenas.
- B II, apenas.
- C I e III, apenas.
- D II e III, apenas.
- E I, II e III.

ENGENHARIA DE COMPUTAÇÃO 2019(GABARITO)

- 3 **P2** Uma árvore binária de busca é uma árvore ordenada que pode apresentar prejuízos no desempenho de determinados algoritmos em função do desbalanceamento causado pela ordem de inserção dos elementos na estrutura. Uma árvore AVL é uma árvore binária de busca balanceada em que a diferença em módulo entre a altura da subárvore esquerda e a altura da subárvore direita de cada nó é, no máximo, de uma unidade.

Nesse contexto, faça o que se pede nos itens a seguir.

- a Apresente uma árvore binária de busca balanceada com os elementos 2, 9, 15, 21, 27, 36 e 50 em que o nó raiz principal contém o elemento 21 e o balanceamento de cada nó seja no máximo uma unidade. (valor: 3,0 pontos)
- b Considerando as inserções dos elementos 9, 27 e 50, nesta ordem, em uma árvore AVL inicialmente vazia, apresente a árvore resultante. (valor: 3,0 pontos)
- c Considerando as inserções dos elementos 9, 27, 50, 15, 2, 21 e 36, nesta ordem, em uma árvore AVL inicialmente vazia, apresente a árvore resultante. (valor: 4,0 pontos)

- 4 **P1** Na matemática, um produtório é definido como:

$$\prod_{i=m}^n x_i = x_m \times x_{m+1} \times x_{m+2} \times \dots \times x_{n-1} \times x_n$$

Com base nessa equação, e considerando que

$$x_i = i + \frac{1}{i}$$

, com

$$i > 0$$

, faça o que se pede nos itens a seguir.

- (a) Escreva uma função iterativa, em linguagem C, que receba os parâmetros de limite inferior m e de limite superior n , calcule e retorne o resultado do produtório. (valor: 5,0 pontos).
- (b) Escreva uma função recursiva, em linguagem C, que receba os parâmetros de limite inferior m e de limite superior n , calcule e retorne o resultado do produtório. (valor: 5,0 pontos).
- 9 P1 O MergeSort é um método de ordenação que combina dois vetores ordenados e cria um terceiro vetor maior também ordenado. O algoritmo abaixo apresenta essa ideia e combina os vetores $a[lo..mid]$ e $a[mid+1..hi]$ no vetor $a[lo..hi]$.

```
public class MergeSort {
    private static Comparable[] aux;
    public static void merge(Comparable[] a, int lo, int mid,
int hi) {
        int i = lo, j = mid+1;
        for (int k = lo; k <= hi; k++)
            aux[k] = a[k];
        for (int k = lo; k <= hi; k++) {
            if (i > mid)
                a[k] = aux[j++];
            else if (j > hi)
                a[k] = aux[i++];
            else if (aux[j].compareTo(aux[i]) < 0)
                a[k] = aux[j++];
            else
                a[k] = aux[i++];
        }
    }
    public static void sort(Comparable[] a) {
        aux = new Comparable[a.length];
        sort(a, 0, a.length - 1);
    }
    private static void sort(Comparable[] a, int lo, int hi) {
        //implementação
    }
}
```

SEdgeWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011 (adaptado).

Considerando o código apresentado, a implementação do protótipo do método sort da classe MergeSort é

A `if (hi == lo)`
 `return;`
 `int mid = lo + (hi - lo)/2;`
 `sort(a, lo, mid);`
 `sort(a, mid, hi);`
 `merge(a, lo, mid, hi);`

B `if (hi > lo)`
 `return;`
 `int mid = lo + (hi - lo)/2;`
 `sort(a, lo, mid);`
 `sort(a, mid, hi);`
 `merge(a, lo, mid, hi);`

C `if (hi <= lo)`
 `return;`
 `int mid = lo + (hi - lo)/2;`
 `sort(a, lo, mid);`
 `sort(a, mid, hi);`
 `merge(a, lo, mid, hi);`

D `if (hi > lo)`
 `return;`
 `int mid = lo + (hi - lo)/2;`
 `sort(a, lo, mid);`
 `sort(a, mid+1, hi);`
 `merge(a, lo, mid, hi);`

E `if (hi <= lo)`
 `return;`
 `int mid = lo + (hi - lo)/2;`
 `sort(a, lo, mid);`
 `sort(a, mid+1, hi);`
 `merge(a, lo, mid, hi);`

- 24 P2** Protocolos de roteamento de estado de enlace utilizam difusão para propagar informações de estado de enlace que são usadas para calcular rotas individuais. Entretanto, algumas técnicas provocam a transmissão

de pacotes redundantes na rede. Idealmente, cada nó deveria receber apenas uma cópia do pacote de difusão.

Uma técnica utilizada para resolver o problema da redundância de pacotes, é a difusão por spanning tree. Uma spanning tree de um grafo $G = (N, E)$ é um grafo $G' = (N, E')$ tal que E' é um subconjunto de E , G' é conexo, não possui ciclos e contém todos os nós originais em G . Se cada enlace tiver um custo associado e o custo de uma árvore for a soma dos custos dos enlaces, então uma árvore cujo custo seja o mínimo entre todas as spanning trees do grafo é denominada uma spanning tree mínima.

KUROSE, J. F.; ROSS, K. W. *Redes de computadores e a Internet: uma abordagem top-down*. 6. ed. São Paulo: Pearson Education do Brasil, 2013 (adaptado).

Considere uma rede composta por 6 roteadores, designados pelas letras A, B, C, D, E e F, conectados conforme a seguinte tabela de custos de seus enlaces:

Conexão	Custo do enlace
A - B	2
A - C	2
B - C	2
B - D	3
C - D	3
C - E	1
C - F	1
D - F	2
E - F	1

Neste cenário, o custo da spanning tree mínima correspondente é, exatamente:

A 5.

B 7.

C 8.

D 9.

E 11.

- 25 P2** A linguagem Python não permite alguns tipos de otimização como, por exemplo, a recursão em cauda e, devido à sua natureza dinâmica, é impossível realizar esse tipo de otimização em tempo de compilação tal como em linguagens funcionais como Haskell ou ML.

Disponível em: <http://www.python-history.blogspot.com/2009/04/origins-of-pythons-functional-features.html>. Acesso: em 15 jun. 2019 (adaptado).

O trecho de código a seguir, escrito em Python, realiza a busca binária de um elemento x em uma lista lst e a função `binary_search` tem código recursivo em cauda.

```
1 def binary_search(x, lst, low=None, high=None):
2     if low == None : low = 0
3     if high == None : high = len(lst)-1
4     mid = low + (high - low) // 2
5     if low > high :
6         return None
7     elif lst[mid] == x :
8         return mid
9     elif lst[mid] > x :
10        return binary_search(x, lst, low, mid-1)
11    else :
12        return binary_search(x, lst, mid+1, high)
```

Disponível em: <https://www.kylem.net/programming/tailcall.html>. Acesso em: 15 jun. 2019 (adaptado).

Considerando esse trecho de código, avalie as afirmações a seguir.

I. Substituindo-se o conteúdo da linha 10 por `high = mid - 1` e substituindo-se o conteúdo da linha 12 por `low = mid + 1`, não se altera o resultado de uma busca.

II. Envolvendo-se o código das linhas 4 a 12 em um laço `while True`, substituindo-se o conteúdo da linha 10 por `high = mid - 1` e substi-

tuindo-se o conteúdo da linha 12 por $\text{low} = \text{mid} + 1$ remove-se a recursão de cauda e o resultado da busca não é alterado.

III. Substituindo-se o código da linha 10 por:

```
newhigh = mid-1  
return binary_search(x, lst, low, newhigh)
```

e substituindo-se o código da linha 12 por:

```
newlow = mid+1  
return binary_search(x, lst, newlow, high)
```

remove-se a recursão de cauda.

IV. Substituindo-se o conteúdo das linhas 9 a 12 por

```
if lst[mid] > x :  
    newlow = low  
    newhigh = mid-1  
else:  
    newlow = mid+1  
    newhigh = high  
return binary_search(x, lst, newlow, newhigh)
```

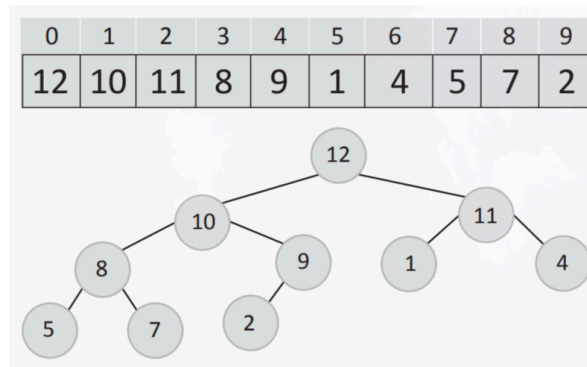
mantém-se o resultado da busca.

É correto o que se afirma em

- ☐ A I, apenas.
- ☐ B II e III, apenas.
- ☐ C II e IV, apenas.
- ☐ D I, III e IV, apenas.
- ☐ E I, II, III e IV.

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO 2021 (GABARITO)

- 5 P2 Um heap binário é um arranjo que pode ser visualizado como uma árvore binária, sendo que cada nó da árvore corresponde a um elemento do arranjo, como pode ser observado na figura a seguir.



Percebe-se que existem dois tipos de heaps: heaps máximo e heaps mínimo. O heap máximo é uma estrutura de dados que possibilita a consulta ou extração de forma eficiente do maior elemento de uma coleção. A propriedade de heap máximo especifica que um nó filho (no código calculado pelas funções `left` e `right`) tem sempre armazenado um valor menor ou igual ao seu pai.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. MIT Press and McGraw-Hill. p. 131-161, 2009 (adaptado).

Considerando a implementação a seguir, o `heapify` é uma função auxiliar para reorganizar o arranjo (garantindo a propriedade de heap máximo em uma determinada posição do arranjo) e `buildHeap` é uma função que usa `heapify` para reorganizar todas as posições do arranjo (garantindo a propriedade de heap máximo para todos os elementos).

```
int left(int i) { return (2 * i + 1); }
int right(int i) { return (2 * i + 2); }
/* a - arranjo, n - número de elementos,
   i - posição do elemento que deve ser
   colocado em propriedade de heap */
void heapify (int *a, int n, int i)
{
    int e, d, max, aux;
    e = left(i);
    d = right(i);
    if (e < n && a[e] > a[i])
```



```

        max = e;
    else
        max = i;
    if (d < n && a[d] > a[max])
        max = d;
    if (max != i)
    {
        aux = a[i];
        a[i] = a[max];
        a[max] = aux;
        heapify(a, n, max);
    }
}
/* a - arranjo, n - número de elementos */
void buildHeap(int *a, int n)
{
    int i;
    for (i = (n-1)/2; i >= 0; i--)
        heapify(a, n, i);
}

```

De acordo com as informações apresentadas, faça o que se pede nos itens a seguir.

- a** Como ficará o arranjo `int a[ ] = {2, 5, 8, 13, 21, 1, 3, 34}` após a execução da função `buildHeap(a, 8)`. (valor: 5,0 pontos)
- b** Apresente a complexidade de tempo no pior caso para a função `heapify`, use a notação O ou Q. (valor: 5,0 pontos)

20 P1 Observe o código abaixo escrito na linguagem C.

```

1  #include <stdio.h>
2  #define TAM 10
3  int funcao1(int vetor[], int v){
4      int i;
5      for (i = 0; i < TAM; i++){
6          if (vetor[i] == v)
7              return i;
8      }
9      return -1;
10 }
11 int funcao2(int vetor[], int v, int i, int f){

```

```
12     int m = (i + f) / 2;
13     if (v == vetor[m])
14         return m;
15     if (i >= f)
16         return -1;
17     if (v > vetor[m])
18         return funcao2(vetor, v, m+1, f);
19     else
20         return funcao2(vetor, v, i, m-1);
21 }
22 int main(){
23     int vetor[TAM] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
24     printf("%d - %d", funcao1(vetor, 15), funcao2(vetor, 15,
25     0, TAM-1));
26     return 0;
27 }
```

A respeito das funções implementadas, avalie as afirmações a seguir.

I. O resultado da impressão na linha 24 é: 7 - 7.

II. A função funcao1, no pior caso, é uma estratégia mais rápida do que a funcao2.

III. A função funcao2 implementa uma estratégia iterativa na concepção do algoritmo.

É correto o que se afirma em

- ☐ A I, apenas.
- ☐ B III, apenas.
- ☐ C I e II, apenas.
- ☐ D II e III, apenas.
- ☐ E I, II e III.

- 23 P2** O uso da estrutura de dados tipo Árvore Binária de Busca é uma técnica fundamental de programação. Uma árvore binária é um conjunto finito de elementos que está vazio ou é particionado em três

subconjuntos, a saber: 1) raiz da árvore - elemento inicial (único), 2) subárvore da esquerda - se vista isoladamente compõe outra árvore e 3) subárvore da direita - se vista isoladamente compõe outra árvore. A árvore pode não ter qualquer elemento (árvore vazia). A definição de árvore é recursiva e, devido a isso, muitas operações sobre árvores binárias utilizam recursão. Sendo “A” a raiz de uma árvore binária e “B” a raiz de sua subárvore esquerda ou direita, é dito que “A” é pai de “B” e que “B” é filho de “A”. Um elemento sem filhos é chamado de folha. A altura da árvore é o número de elementos encontrados no caminho descendente mais longo que liga a sua raiz até uma folha.

Uma Árvore de Busca Binária é uma árvore binária especializada, na qual a informação que o elemento filho esquerdo possui é numericamente menor que a informação do elemento pai. De forma análoga, a informação que o elemento filho direito possui é numericamente maior ou igual à informação do elemento pai. O objetivo de organizar dados em Árvores Binárias de Busca é facilitar a tarefa de encontrar um determinado elemento. O percurso completo de uma árvore binária consiste em visitar todos os elementos desta árvore, segundo algum critério, a fim de processá-los. Três formas são bem conhecidas para a realização deste percurso: 1) pré-ordem, 2) em-ordem e 3) pós-ordem. A figura a seguir mostra um exemplo de árvore binária.

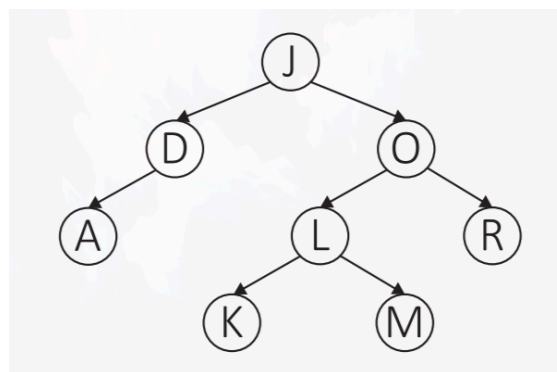


Figura – Exemplo de Árvore Binária

LAUREANO, M. A. P. *Estrutura de Dados com Algoritmos*. São Paulo: Brasport, 2008. p. 126, 129, 136 (adaptado).

Considerando o texto e a figura apresentados e que a seguinte lista de elementos numéricos: (27, 34, 40, 18, 23, 5, 25, 36, 10, 7, -2) seja totalmente transferida para uma estrutura de Árvore Binária de Busca,

inicialmente vazia, elemento a elemento, da esquerda para a direita, assinale a alternativa correta.

- (A)** A árvore resultante terá 5 níveis de altura, com 6 elementos à esquerda da raiz principal (inicial) e 4 elementos à direita.
 - (B)** O percurso da árvore em Pré-ordem irá processar os elementos na seguinte ordem (do primeiro ao último): $-2, 7, 10, 5, 25, 23, 18, 36, 40, 34, 27$.
 - (C)** O percurso da árvore em Em-ordem irá processar os elementos na seguinte ordem (do primeiro ao último): $-2, 5, 7, 10, 18, 23, 25, 27, 34, 36, 40$.
 - (D)** O percurso da árvore em Pós-ordem irá processar os elementos na seguinte ordem (do primeiro ao último): $27, 18, 5, -2, 10, 7, 23, 25, 34, 40, 36$.
 - (E)** O número máximo de elementos que essa árvore poderá ter com 10 níveis será de 1 024 elementos.
- (32) P1** Existe um grande número de implementações para algoritmos de ordenação. Um dos fatores a serem considerados, por exemplo, é o número máximo e médio de comparações que são necessárias para ordenar um vetor com n elementos. Diz-se também que um algoritmo de ordenação é estável se ele preserva a ordem de elementos que são iguais. Isto é, se tais elementos aparecem na sequência ordenada na mesma ordem em que estão na sequência inicial. Analise o algoritmo abaixo, onde A é um vetor e “ i, j, lo e hi ” são índices do vetor:

```
algoritmo ordena(A, lo, hi)
    se lo < hi então
        p := particao(A, lo, hi)
        ordena(A, lo, p - 1)
        ordena(A, p + 1, hi)
```

```
algoritmo particao(A, lo, hi)
    pivot := A[hi]
    i := lo
    repita para j := lo até hi
```

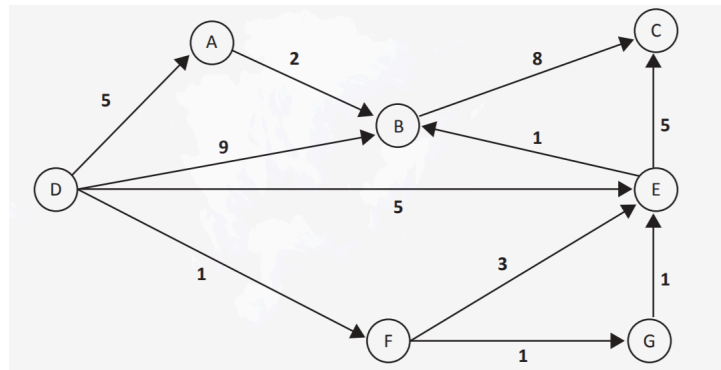
```
    se A[j] < pivot entao
        troca A[i] com A[j]
        i := i + 1
    troca A[i] com A[hi]
    return i
```

Com relação ao algoritmo apresentado, avalie as afirmações a seguir.

- I. O algoritmo precisa de um espaço adicional $O(n)$ para a pilha de recursão.
- II. O algoritmo apresentado é um algoritmo de ordenação recursivo e estável.
- III. O algoritmo precisa, em média, de $O(n \log n)$ comparações para ordenar n itens.
- IV. O uso do primeiro elemento do vetor como “pivot” é mais eficiente que usar o último.

É correto apenas o que se afirma em

- ☐ A I e III.
 - ☐ B II e IV.
 - ☐ C III e IV.
 - ☐ D I, II e III.
 - ☐ E I, II e IV.
-
- 34 P2** O algoritmo de Dijkstra para o problema do caminho mínimo em dígrafos com pesos utiliza uma fila de prioridades de vértices, na qual as prioridades são uma estimativa do custo final. A cada iteração, um vértice é retirado da fila, e os arcos que começam nesse vértice são analisados. Considere o seguinte grafo, no qual deseja-se conhecer o custo de um caminho mínimo para cada vértice, a partir do vértice D. Considere que -1 representa um custo “infinito”, ou seja, nenhum caminho até o vértice foi até o momento descoberto.



Com base nas informações e no grafo apresentados, assinale a alternativa que representa a estimativa de custo após duas iterações do algoritmo.

- ☒ A: 5 B: 6 C: 10 D: 0 E: 4 F: 1 G: -1
- ☐ B: 5 B: 9 C: -1 D: 0 E: 5 F: 1 G: -1
- ☐ C: 5 B: 9 C: -1 D: 0 E: 4 F: 1 G: 2
- ☐ D: 5 B: 7 C: 8 D: 0 E: 4 F: 1 G: 2
- ☐ E: 5 B: 6 C: 8 D: 0 E: 3 F: 1 G: 2

ENGENHARIA DE COMPUTAÇÃO 2023(GABARITO)

- ② **P1** Considere que uma rede de varejo resolva consolidar a base de dados de suas M lojas, o que resulta em uma tabela X não ordenada com N registros de clientes, possivelmente repetidos. Devido à necessidade de se criar uma tabela Y contendo os clientes da tabela X com as repetições eliminadas, cogita-se utilizar dois possíveis algoritmos, os quais são apresentados a seguir.

Algoritmo A, que executa as seguintes ações:

1. Cria uma tabela Y inicialmente vazia;
2. Percorre a tabela X , cliente por cliente, verificando se cada um deles já está na tabela Y . Caso não esteja, insere na tabela Y o cliente que está faltando.

Algoritmo B, que executa as seguintes ações:

1. Cria uma tabela Y inicialmente vazia;
2. Ordena a tabela X usando o algoritmo quicksort;
3. Insere, na tabela Y , o cliente da primeira posição da tabela X ;
4. Percorre a tabela X , cliente por cliente, a partir do segundo cliente, verificando se cada cliente é igual ao anterior. Caso não seja, insere o cliente na tabela Y .

Como resultado das ações tanto do algoritmo A quanto do algoritmo B, a tabela Y gerada ao final conterá os clientes da tabela X com as repetições eliminadas.

A partir dessas informações, observe o código em linguagem C apresentado a seguir.

```
void quicksort(int *v, int ini, int fim) {
    // v é o vetor a ser ordenado
    // ini é o índice do primeiro elemento a ser ordenado
    // fim é o índice do último elemento a ser ordenado
    if(ini < fim) {
        x = particiona(v, ini, fim, (ini+fim)/2);
        // atribuir os valores para a, b, c e d
        quicksort(v,a,b);
        quicksort(v,c,d);
    }
}

int particiona (int *vetor, int ini, int fim, int pivot){
```

```
// implementar uma função que reorganiza o vetor e retorna
a posição
// final x do pivot. Ao final do processo, os elementos
menores ou iguais a
// vetor[pivot] devem ter índice menor do que x; os
elementos maiores que
// vetor[pivot] devem ter índice maior do que x.
}

void troca (int *vetor, int i, int j) {
    // função auxiliar que permuta os conteúdos das posições i
e j do vetor
    aux = vetor[i];
    vetor[i] = vetor[j];
    vetor[j] = aux;
}
```

Com base nos dados apresentados, faça o que se pede nos itens a seguir.

- a** Determine o número máximo de comparações executadas no passo 2 do algoritmo A. (valor: 2,0 pontos)
 - b** Determine o número máximo de comparações executadas no passo 4 do algoritmo B. (valor: 2,0 pontos)
 - c** Na implementação recursiva do programa apresentado, quais são os valores dos parâmetros a, b, c e d? (valor: 2,0 pontos)
 - d** Escreva o corpo da função particiona em linguagem C utilizando a função auxiliar troca, a qual foi definida anteriormente. (valor: 4,0 pontos)
-
- 15 P1** Memory leak, ou vazamento de memória, é um problema que ocorre em sistemas computacionais quando uma parte da memória, alocada para uma determinada operação, não é liberada quando se torna desnecessária. Na linguagem C, esse tipo de problema é quase sempre relacionado ao uso incorreto das funções malloc() e free(). Esse erro de programação pode levar a falhas no sistema se a memória for completamente consumida.

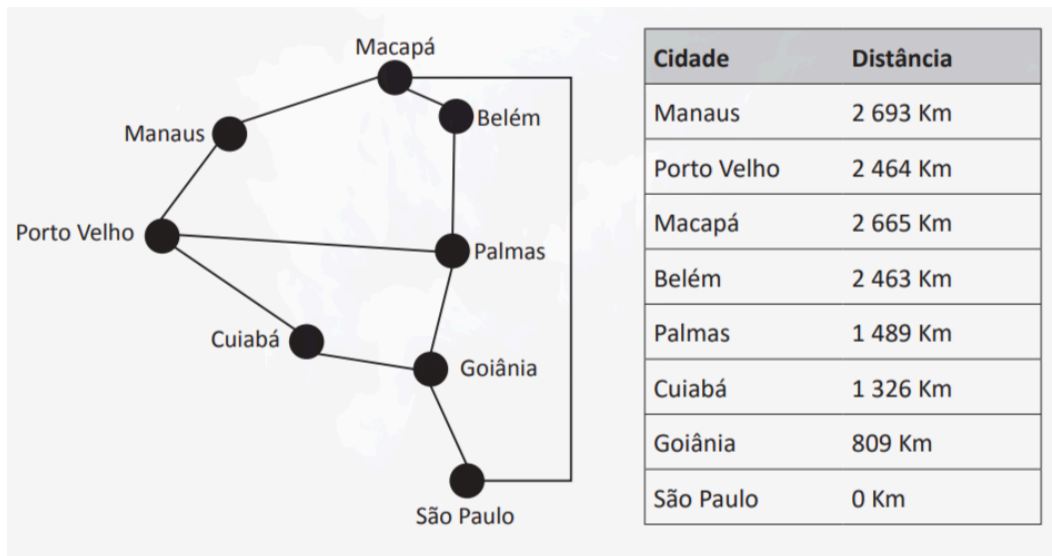
A partir dessas informações, assinale a opção que apresenta um trecho com memory leak.

- (A) `void f(){
 void *s;
 s = malloc(50);
 free(s);
}`
- (B) `int f(){
 float *a;
 return 0;
}`
- (C) `int f(char *data){
 void *s;
 s = malloc(50);
 int size = strlen(data);
 if (size > 50)
 return(-1);
 free(s);
 return 0;
}`
- (D) `int *f(int n){
 int *num = malloc(sizeof(int)*n);
 return num;
}
int main(void){
 int *num;
 num = f(10);
 free(num);
 return 0;
}`
- (E) `void f(int n){
 char *m = malloc(10);
 char *n = malloc(10);
 free(m);
 m = n;
 free(m);
 free(n);
}`

- 18 P1** Vetores de tamanho dinâmico são muito utilizados como estruturas de dados para armazenar listas e tabelas de dispersão (tabelas hash). Essa representação permite que o número máximo de elementos a ser inserido não precise ser pré-determinado. Uma técnica para implementar vetores de tamanho dinâmico é aquela que dobra o tamanho do vetor quando o número de itens a ser armazenado ultrapassa o tamanho atual do vetor. Essa operação requer uma alocação de memória para conter o vetor com o dobro do tamanho atual e a realização de cópia dos elementos para o novo vetor. Com base nessas informações, considere uma lista linear implementada com um vetor dinâmico. Assuma que todos os itens inseridos na lista tenham o mesmo tamanho e que o vetor tenha tamanho inicial para comportar apenas 1 item.

Considerando uma sequência de n inserções no final dessa lista, o tempo médio por inserção e o tempo total gasto para executar todas as n operações são, respectivamente, da ordem de

- A** $\Theta(\log n)$ e $\Theta(n \log n)$
 - B** $\Theta(1)$ e $\Theta(n)$
 - C** $\Theta(1)$ e $\Theta(n^2)$
 - D** $\Theta(n)$ e $\Theta(n^2)$
 - E** $\Theta(\sqrt{n})$ e $\Theta(n^{\frac{3}{2}})$
- 32 P2** Suponha que uma empresa de transportes deseje calcular uma rota de Manaus para São Paulo utilizando a estratégia da busca gulosa, técnica de busca local que seleciona a melhor alternativa disponível a cada passo. Observe, a seguir, um grafo de conexão de cidades até o destino final e o quadro com estimativas de distância de São Paulo até as demais cidades do grafo.



Com base nessas informações, é correto afirmar que a solução encontrada a partir da utilização do algoritmo será

- (A) Manaus → Macapá → São Paulo
- (B) Manaus → Porto Velho → Cuiabá → Goiânia → São Paulo
- (C) Manaus → Porto Velho → Palmas → Goiânia → São Paulo
- (D) Manaus → Macapá → Belém → Palmas → Goiânia → São Paulo
- (E) Manaus → Macapá → Belém → Palmas → Porto Velho → Cuiabá → Goiânia → São Paulo

- 34 **P2** Cada roteador em redes de computadores precisa implementar alguma estratégia de enfileiramento para controlar como os pacotes são armazenados em buffer enquanto esperam para serem transmitidos, independentemente do mecanismo de alocação de recursos. O algoritmo de enfileiramento aloca tanto largura de banda, ao transmitir pacotes, quanto espaço de buffer, ao decidir quais pacotes são descartados.

PETERSON, L. L.; DAVIE, B. S. *Redes de Computadores: uma abordagem de sistemas*. Rio de Janeiro: Elsevier, 2013 (adaptado).

Considerando as informações apresentadas, avalie as afirmações a seguir.

I. O algoritmo FIFO (First-In, First-Out) é adequado para situações em que o tráfego de dados com rajadas de longa duração provoca descarte de pacotes.

II. O algoritmo de enfileiramento justo ponderado (WFQ, do inglês Weighted Fair Queuing) permite definir um peso para cada fila, definindo quantos bits são transmitidos sempre que o roteador atender a uma determinada fila.

III. O algoritmo de enfileiramento por prioridade (PQ, do inglês Priority Queuing) evita que uma fila de menor prioridade fique indefinidamente sem ser atendida (starvation), utilizando o algoritmo Round-Robin para servir a todas as filas.

IV. Os roteadores que implementam o algoritmo de detecção antecipada aleatória (RED, do inglês Random Early Detection) mantêm a média acumulada do tamanho de suas filas e, quando esse tamanho ultrapassa, em algum enlace, um determinado limiar, uma fração dos pacotes é descartada aleatoriamente.

É correto apenas o que se afirma em

- ☐ A I e III.
- ☐ B I e IV.
- ☐ C II e IV.
- ☐ D I, II e III.
- ☐ E II, III e IV.

EXERCÍCIOS SABOR ENADE

MÚLTIPLA ESCOLHA

- 35 Tipos Abstratos de Dados (TADs) são fundamentais para o desenvolvimento de software modular e reutilizável. Um TAD define um conjunto de operações sobre um tipo de dado, ocultando a representação interna dos dados e permitindo que implementações diferentes sejam intercambiáveis sem afetar o código cliente. Essa propriedade é conhecida como encapsulamento e é um dos pilares da programação orientada a objetos, embora também possa ser aplicada em linguagens procedurais como C.

Considere que uma biblioteca de estruturas de dados oferece um TAD `ListaOrdenada` com as seguintes operações na sua interface pública:

```
typedef struct lista ListaOrdenada;

ListaOrdenada* lista_criar();
void lista_inserir(ListaOrdenada *l, int valor);    // mantém
ordenado
int lista_buscar(ListaOrdenada *l, int valor);      // retorna
1 se encontrou, 0 caso contrário
void lista_remove(ListaOrdenada *l, int valor);
int lista_tamanho(ListaOrdenada *l);
void lista_destruir(ListaOrdenada *l);
```

Dois programadores implementam esse TAD de formas diferentes: o primeiro utiliza um vetor dinâmico; o segundo utiliza uma lista simplesmente encadeada com inserção ordenada.

Um cliente utiliza o TAD para armazenar identificadores de produtos em um sistema de estoque. Inicialmente, o sistema contém 1000 produtos cadastrados. O cliente executa as seguintes operações:

1. Busca pelo produto com identificador 500.
2. Insere um novo produto com identificador 750.
3. Remove o produto com identificador 200.

Assinale a alternativa que apresenta corretamente as complexidades de tempo dessas operações na implementação com **lista encadeada**, considerando que a busca sempre percorre a lista do início.

- A Busca: $O(1)$, Inserção: $O(n)$, Remoção: $O(n)$

- B** Busca: $O(n)$, Inserção: $O(n)$, Remoção: $O(n)$
 - C** Busca: $O(n)$, Inserção: $O(1)$, Remoção: $O(1)$
 - D** Busca: $O(\log n)$, Inserção: $O(n)$, Remoção: $O(n)$
 - E** Busca: $O(n)$, Inserção: $O(n \log n)$, Remoção: $O(n)$
- 36** Sistemas de edição de texto, como processadores de palavras e ambientes de desenvolvimento integrado (IDEs), frequentemente implementam a funcionalidade de “desfazer” (*undo*) por meio de uma pilha que armazena o histórico de operações realizadas pelo usuário. A cada ação executada — digitação, exclusão, formatação —, um registro da operação é empilhado; quando o usuário solicita “desfazer”, a operação mais recente é desempilhada e revertida. Essa arquitetura garante que as ações sejam desfeitas na ordem inversa em que foram realizadas, preservando a integridade do documento.

Um desenvolvedor implementou uma pilha genérica em C utilizando um vetor de ponteiros para `void`, permitindo armazenar qualquer tipo de dado. A estrutura e algumas operações estão definidas abaixo:

```
typedef struct {
    void **dados;
    int topo;
    int capacidade;
} Pilha;

Pilha* criar_pilha(int cap) {
    Pilha *p = malloc(sizeof(Pilha));
    p->dados = malloc(cap * sizeof(void*));
    p->topo = -1;
    p->capacidade = cap;
    return p;
}

int empilhar(Pilha *p, void *elem) {
    if (p->topo == p->capacidade - 1) return -1;
```

```
p->dados[++(p->topo)] = elem;
return 1;
}

void* desempilhar(Pilha *p) {
    if (p->topo == -1) return -1;
    return p->dados[(p->topo)--];
}
```

A respeito dessa implementação, avalie as seguintes afirmações.

- I. A operação empilhar possui complexidade de tempo $O(1)$ no pior caso, independentemente do tipo de dado armazenado.
- II. Quando `p->topo` vale `-1`, a pilha está cheia, pois não há posições disponíveis no vetor.
- III. A liberação completa da memória alocada para a pilha requer duas chamadas a `free`: uma para o vetor `dados` e outra para a estrutura `Pilha`.
- IV. Se a pilha for redimensionada dinamicamente quando estiver cheia (dobrando a capacidade), a complexidade por operação de empilhar permanece, na média, $O(1)$.

É correto apenas o que se afirma em

- ☐ A I e III.
- ☐ B I e IV.
- ☐ C II e III.
- ☐ D II, III e IV.
- ☐ E I, III e IV.

- 37 Em sistemas operacionais, o mecanismo de chamadas de função depende de uma estrutura de dados fundamental para controlar o fluxo

de execução: a pilha de execução (*call stack*). Toda vez que uma função é invocada, um novo *stack frame* é empilhado contendo o endereço de retorno, os parâmetros e as variáveis locais. Ao retornar, o *frame* é desempilhado e o controle volta ao ponto de chamada.

Considere o trecho de código C a seguir, que manipula uma pilha implementada com vetor de capacidade máxima 4 e topo inicializado em -1 :

```
void push(int *pilha, int *topo, int val) {
    if (*topo == 3) return -1;          // pilha cheia
    pilha[++(*topo)] = val;
}

int pop(int *pilha, int *topo) {
    if (*topo == -1) return -1;        // pilha vazia
    return pilha[(*topo)--];
}

int main() {
    int p[4], t = -1;
    push(p, &t, 10);
    push(p, &t, 20);
    push(p, &t, 30);
    pop(p, &t);
    push(p, &t, 40);
    push(p, &t, 50);
    push(p, &t, 60);    // (*)
    printf("%d %d\n", pop(p, &t), t);
    return 0;
}
```

Avalie as afirmações a seguir sobre o comportamento desse programa.

- I. Na linha marcada com (*), a operação de `push` com valor 60 é ignorada porque a pilha já está cheia.
- II. O valor impresso pela chamada a `printf` é 50 2.
- III. Após a execução completa do `main`, o vetor `p` contém, nas posições 0 a 3, os valores 10, 20, 30, 50, nessa ordem.
- IV. Se a pilha tivesse capacidade 5, o valor impresso seria 60 3.

É correto apenas o que se afirma em

- (A) I e III.
 - (B) II e IV.
 - (C) I, II e III.
 - (D) I, II e IV.
 - (E) I, III e IV.
- 38 Calculadoras científicas modernas frequentemente utilizam a notação polonesa reversa (RPN — *Reverse Polish Notation*) para avaliar expressões aritméticas, eliminando a necessidade de parênteses e simplificando o processamento. Nessa notação, os operandos aparecem antes do operador correspondente; por exemplo, a expressão infixa $(3 + 4) * 5$ torna-se $3\ 4\ +\ 5\ *$ em RPN.

A avaliação de expressões RPN é naturalmente implementada utilizando uma pilha: cada operando é empilhado, e quando um operador é encontrado, os dois operandos do topo são desempilhados, a operação é executada e o resultado é empilhado novamente. Segundo SEDGEWICK e WAYNE (*Algorithms*, Addison-Wesley), essa estratégia garante avaliação em tempo linear (adaptado).

Considere a seguinte implementação em C de uma calculadora RPN que processa expressões com números inteiros de um dígito (0–9) e os operadores +, -, * e /:

```
typedef struct {
    int dados[100];
    int topo;
} Pilha;

void init(Pilha *p) { p->topo = -1; }
void push(Pilha *p, int v) { p->dados[++(p->topo)] = v; }
int pop(Pilha *p) { return p->dados[(p->topo)--]; }
int vazia(Pilha *p) { return p->topo == -1; }
```

```
int avaliar_rpn(char *expr) {
    Pilha p;
    init(&p);
    for (int i = 0; expr[i] != '\0'; i++) {
        char c = expr[i];
        if (c >= '0' && c <= '9') {
            push(&p, c - '0');
        } else if (c == '+' || c == '-' || c == '*' || c ==
        '/') {
            int b = pop(&p); // operando direito
            int a = pop(&p); // operando esquerdo
            int res;
            switch (c) {
                case '+': res = a + b; break;
                case '-': res = a - b; break;
                case '*': res = a * b; break;
                case '/': res = a / b; break;
            }
            push(&p, res);
        }
    }
    return pop(&p);
}
```

Avalie as afirmações a seguir sobre essa implementação.

- I. A avaliação de uma expressão RPN válida com n tokens (operandos e operadores) possui complexidade de tempo $\Theta(n)$, pois cada token é processado exatamente uma vez com operações de pilha $O(1)$.
- II. Se a expressão de entrada for 9 9 9 9 9 9 9 9 9 + + + + + + + + + (dez noves seguidos de nove adições), ocorrerá *stack overflow* porque a pilha implementada suporta no máximo 100 elementos.
- III. A estrutura de pilha é suficiente para avaliar qualquer expressão RPN válida; não são necessárias estruturas auxiliares como filas ou listas encadeadas.
- IV. Na operação de subtração e divisão, a ordem dos operandos desempilhados não afeta o resultado final, pois as operações aritméticas são comutativas.

É correto apenas o que se afirma em

- A** I e III.

B II e IV.

C I, II e III.

D I, III e IV.

E II, III e IV.

- 39** No desenvolvimento de um sistema de gerenciamento de pedidos para um e-commerce, a equipe de engenharia optou por utilizar uma lista simplesmente encadeada para armazenar os pedidos em aberto, uma vez que inserções e remoções ocorrem com frequência e a quantidade de pedidos varia dinamicamente. Cada nó da lista armazena o identificador do pedido e um ponteiro para o próximo nó.

Analise o trecho de código C a seguir, que tenta remover o nó com um determinado valor da lista:

```
typedef struct No {
    int valor;
    struct No *prox;
} No;

void remove_valor(No **lista, int alvo) {
    No *atual = *lista;
    No *anterior = NULL;

    while (atual != NULL && atual->valor != alvo) {
        anterior = atual;
        atual = atual->prox;
    }

    if (atual == NULL) return;

    if (anterior == NULL)
        *lista = atual->prox;
    else
        anterior->prox = atual->prox;
```

```
    free(atual);  
}
```

Considere uma lista com os nós [5] -> [12] -> [7] -> [3] -> NULL e avalie as afirmações a seguir sobre a função `remove_valor`.

I. Ao chamar `remove_valor(&lista, 5)`, o ponteiro `*lista` é atualizado corretamente para apontar para o nó de valor 12, e o nó removido é liberado com `free`.

II. Ao chamar `remove_valor(&lista, 3)`, o nó com valor 3 é removido e o ponteiro `prox` do nó anterior (valor 7) passa a apontar para NULL.

III. Se o valor alvo não existir na lista, a função causa comportamento indefinido por tentar liberar um ponteiro nulo.

IV. A função aceita o endereço do ponteiro de cabeça (No `**lista`) para permitir que o nó cabeça seja removido sem necessidade de um nó sentinela.

É correto apenas o que se afirma em

☐ A I e III.

☐ B II e III.

☐ C I, II e III.

☐ D I, II e IV.

☐ E I, III e IV.

- ☐ 40 Sistemas de arquivos frequentemente utilizam algoritmos recursivos para calcular o tamanho total de diretórios, percorrendo subdiretórios aninhados. A recursão é natural para essa tarefa porque a estrutura de diretórios é hierárquica e auto-similar: cada subdiretório pode conter arquivos e outros subdiretórios, exigindo o mesmo procedimento de processamento.

Considere a seguinte função recursiva em C que calcula a soma dos elementos de um vetor:

```
int soma_vetor(int *v, int n) {  
    if (n == 0) return 0;  
    return v[n-1] + soma_vetor(v, n-1);  
}
```

Avalie as afirmações a seguir sobre essa implementação.

- I. Para processar um vetor de n elementos, a função realiza exatamente n chamadas recursivas, uma para cada elemento do vetor.
- II. A recursão sempre é mais eficiente que a solução iterativa equivalente, pois elimina a necessidade de variáveis de controle.
- III. A quantidade de memória auxiliar utilizada é constante, pois não são alocados vetores ou estruturas adicionais.
- IV. A função processa os elementos do vetor da posição $n - 1$ até a posição 0, em ordem inversa.

É correto apenas o que se afirma em

- ☐ A I e II.
- ☐ B II e III.
- ☐ C I e IV.
- ☐ D I, II e III.
- ☐ E II, III e IV.

- 41** Algoritmos recursivos são frequentemente utilizados em jogos eletrônicos para gerar labirintos ou calcular caminhos. A recursão permite dividir problemas complexos em subproblemas menores e mais fáceis de resolver.

Considere a seguinte função recursiva em C:

```
int f(int n) {  
    if (n == 0) return 0;  
    return 1 + f(n / 2);  
}
```

Qual é o valor retornado pela chamada `f(16)`?

(A) 5

(B) 4

(C) 8

(D) 16

(E) 0

- 42 Em redes de computadores, roteadores utilizam filas para armazenar pacotes que aguardam transmissão. Quando um roteador recebe pacotes mais rapidamente do que consegue transmiti-los, os pacotes são enfileirados e processados na ordem de chegada (FIFO — *First In, First Out*). Uma implementação eficiente de filas em sistemas embarcados frequentemente emprega um vetor circular, que permite reutilizar posições liberadas sem a necessidade de deslocar elementos.

Segundo KUROSE e ROSS (*Redes de Computadores e a Internet*, Pearson), o tamanho da fila de um roteador influencia diretamente o atraso de enfileiramento e a probabilidade de perda de pacotes (adaptado). Considere a seguinte implementação de uma fila circular em C:

```
#define MAX 5  
  
typedef struct {  
    int pacotes[MAX];  
    int frente, tras;  
} FilaRoteador;  
  
void inicializar(FilaRoteador *f) {  
    f->frente = f->tras = 0;  
}
```

```
}

int enfileirar(FilaRoteador *f, int pacote) {
    if ((f->tras + 1) % MAX == f->frente)
        return 0; // fila cheia
    f->pacotes[f->tras] = pacote;
    f->tras = (f->tras + 1) % MAX;
    return 1;
}

int desenfileirar(FilaRoteador *f) {
    if (f->frente == f->tras)
        return -1; // fila vazia
    int pacote = f->pacotes[f->frente];
    f->frente = (f->frente + 1) % MAX;
    return pacote;
}
```

Um analista de redes executa a seguinte sequência de operações sobre uma fila recém-inicializada:

```
enfileirar(f, 100);
enfileirar(f, 200);
enfileirar(f, 300);
desenfileirar(f);
enfileirar(f, 400);
enfileirar(f, 500);
```

Assinale a alternativa que apresenta corretamente o estado final da fila.

- A** frente = 1, tras = 0, e o vetor contém 100, 200, 300, 400, 500 nas posições 0 a 4.
- B** frente = 1, tras = 5, e o vetor contém 100, 200, 300, 400, 500 nas posições 0 a 4.
- C** frente = 1, tras = 0, e o vetor contém 400, 500, 300, _, _ nas posições 0 a 4, onde _ indica valor irrelevante.
- D** frente = 0, tras = 4, e o vetor contém 200, 300, 400, 500, _ nas posições 0 a 4.
- E** frente = 1, tras = 0, e o vetor contém _, 200, 300, 400, 500 nas posições 0 a 4, onde _ indica valor irrelevante.

- 43) Sistemas de impressão em rede utilizam filas para gerenciar documentos enviados por múltiplos usuários. Quando um documento é enviado para a impressora, ele é enfileirado e processado na ordem de chegada (FIFO — *First In, First Out*).

Considere a seguinte implementação de uma fila utilizando lista encadeada:

```
estrutura No:
    inteiro valor
    No prox
```

```
estrutura Fila:
    No inicio
    No fim
```

```
função enfileirar(Fila f, inteiro v):
    novo = criar No
    novo.valor = v
    novo.prox = NULO
    se f.fim != NULO então
        f.fim.prox = novo
    f.fim = novo
    se f.inicio == NULO então
        f.inicio = novo
```

```
função desenfileirar(Fila f) retorna inteiro:
    se f.inicio == NULO então
        retornar -1
    temp = f.inicio
    v = temp.valor
    f.inicio = f.inicio.prox
    se f.inicio == NULO então
        f.fim = NULO
    liberar temp
    retornar v
```

Após executar: `enfileirar(f, 10)`, `enfileirar(f, 20)`, `enfileirar(f, 30)`, `desenfileirar(f)`, `enfileirar(f, 40)`, qual é o valor retornado pelo próximo `desenfileirar(f)`?

- A) 20

(B) 10

(C) 30

(D) 40

(E) -1

- 44 Filas circulares implementadas com vetores são amplamente utilizadas em sistemas embarcados devido à sua eficiência de memória. Em uma fila circular, quando o ponteiro de fim alcança o final do vetor, ele “dá a volta” e utiliza as posições liberadas no início.

Considere uma fila circular implementada com as seguintes operações:

```
constante inteiro TAM = 5
```

```
vetor dados[TAM]
```

```
inteiro frente, tras
```

```
função inicializar():
```

```
    frente = 0
```

```
    tras = 0
```

```
função enfileirar(inteiro v):
```

```
    se (tras + 1) mod TAM == frente então
```

```
        retornar "fila cheia"
```

```
    dados[tras] = v
```

```
    tras = (tras + 1) mod TAM
```

```
função desenfileirar() retorna inteiro:
```

```
    se frente == tras então
```

```
        retornar "fila vazia"
```

```
    v = dados[frente]
```

```
    frente = (frente + 1) mod TAM
```

```
    retornar v
```

Inicialmente: frente = 0, tras = 0

Após a sequência de operações:

- enfileirar(100) → enfileirar(200) → enfileirar(300)

- desenfileirar()
- enfileirar(400) → enfileirar(500)

Quais são os valores de frente e tras, respectivamente?

A 1 e 0

B 0 e 5

C 1 e 5

D 3 e 0

E 0 e 0

- 45** Aplicativos de edição de fotos para dispositivos móveis precisam processar imagens de alta resolução com restrições severas de memória. Um desenvolvedor criou uma função para aplicar um filtro de suavização em blocos de pixels representados por uma matriz alocada dinamicamente. O código-fonte está reproduzido abaixo:

```
typedef struct {
    int **pixels;
    int largura, altura;
} Imagem;

Imagem* carregar_imagem(int w, int h) {
    Imagem *img = malloc(sizeof(Imagem));
    img->pixels = malloc(h * sizeof(int*));
    for (int i = 0; i < h; i++)
        img->pixels[i] = malloc(w * sizeof(int));
    img->largura = w;
    img->altura = h;
    return img;
}

void aplicar_filtro(Imagem *img) {
    int **aux = malloc(img->altura * sizeof(int*));
    for (int i = 0; i < img->altura; i++)
```

```
        aux[i] = malloc(img->largura * sizeof(int));

// processamento dos pixels...

for (int i = 0; i < img->altura; i++)
    free(aux[i]);
free(aux);
}

void processar_galeria(int n) {
    Imagem *galeria[n];
    for (int i = 0; i < n; i++) {
        galeria[i] = carregar_imagem(1920, 1080);
        aplicar_filtro(galeria[i]);
        salvar_arquivo(galeria[i]);
        free(galeria[i]);
    }
}
```

Durante a revisão de código, um colega identificou problemas de gerenciamento de memória. Assinale a alternativa que descreve corretamente o(s) vazamento(s) de memória presente(s) no código.

- A** Não há vazamento de memória, pois toda memória alocada em `carregar_imagem` é liberada em `processar_galeria` e toda memória alocada em `aplicar_filtro` é liberada antes do retorno.
- B** Há vazamento de memória em `processar_galeria`, pois `free(galeria[i])` libera apenas a estrutura `Imagem`, mas não libera o vetor de ponteiros `img->pixels` nem os vetores de inteiros alocados para cada linha da matriz.
- C** Há vazamento de memória em `aplicar_filtro`, pois o vetor `aux` não é liberado ao final da função, consumindo memória crescente a cada chamada.
- D** Há vazamento de memória porque `malloc(h * sizeof(int*))` deveria ser `malloc(h * sizeof(int))`, causando alocação insuficiente e corrupção de memória.
- E** Há vazamento de memória porque o vetor `galeria` não é liberado com `free(galeria)` ao final da função `processar_galeria`.

46 Um portal de notícias processa milhões de acessos diários e precisa identificar rapidamente artigos duplicados em sua base de dados. O tempo de execução do algoritmo de deduplicação é crítico para a operação em tempo real do sistema. A equipe de engenharia analisa três algoritmos candidatos:

- Algoritmo A: complexidade $\Theta(n^2)$ no pior caso.
- Algoritmo B: complexidade $\Theta(n \log n)$ no pior caso.
- Algoritmo C: complexidade $\Theta(n)$ no caso médio, mas $\Theta(n^2)$ no pior caso.

O sistema atual processa aproximadamente $n = 10^5$ artigos por hora. A equipe estima que cada operação básica (comparação de identificadores) consome cerca de 1mus (10^{-6} segundos).

Para atender ao requisito de processar todos os artigos em menos de 1 hora, o engenheiro responsável deve escolher o algoritmo adequado.

I. O Algoritmo A consumiria aproximadamente 10^5 segundos para processar $n = 10^5$ artigos, o que excede o limite de 1 hora (3600 segundos).

II. O Algoritmo B é sempre preferível ao Algoritmo A, independentemente do tamanho da entrada, pois $\Theta(n \log n)$ cresce mais lentamente que $\Theta(n^2)$.

III. O Algoritmo C pode ser adequado se a probabilidade de o pior caso ocorrer for muito baixa, mas exige garantias sobre a distribuição dos dados de entrada.

IV. A notação assintótica Θ fornece limites superior e inferior justos, permitindo prever com precisão o tempo absoluto de execução para qualquer valor de n .

É correto apenas o que se afirma em

A I e III.

B II e IV.

C I, II e III.

D I, III e IV.

E II, III e IV.

- 47** A ordenação de dados é uma operação fundamental em sistemas de gerenciamento de informações. Diferentes algoritmos de ordenação apresentam características distintas quanto à eficiência, estabilidade e uso de memória auxiliar.

Considere os seguintes algoritmos de ordenação:

- **Bubblesort**: percorre o vetor repetidamente, trocando elementos adjacentes fora de ordem.
- **Mergesort**: divide o vetor ao meio, ordena recursivamente cada metade e intercala.
- **Quicksort**: escolhe um pivô, particiona o vetor e ordena recursivamente as partições.
- **Selectionsort**: encontra o menor elemento e o coloca na posição correta, repetindo.
- **Insertionsort**: insere cada elemento na posição correta entre os já ordenados.

Assinale a alternativa que associa corretamente cada algoritmo à sua característica.

- A** Bubblesort é o mais eficiente para vetores grandes; Mergesort usa memória auxiliar constante.
- B** Quicksort tem complexidade $O(n^2)$ no melhor caso; Selectionsort é estável.
- C** Mergesort tem complexidade $O(n \log n)$ em todos os casos; Bubblesort tem complexidade $O(n^2)$ em todos os casos.
- D** Insertionsort é eficiente para vetores quase ordenados; Quicksort não usa recursão.

- E** Selectionsort faz $O(n^2)$ comparações no pior caso; Mergesort ordena in-place.

- 48** Um sistema de processamento de transações bancárias precisa ordenar registros de movimentação para geração de extratos. O volume médio é de 1000 registros por consulta.

O desenvolvedor está avaliando dois algoritmos:

- **Algoritmo X:** Insertionsort
- **Algoritmo Y:** Quicksort

Considere as seguintes afirmações sobre a escolha do algoritmo:

- I. Insertionsort é preferível quando os dados já estão quase ordenados.
- II. Quicksort sempre é mais rápido que Insertionsort para qualquer entrada.
- III. Insertionsort tem complexidade $O(n)$ no melhor caso (vetor já ordenado).
- IV. Quicksort utiliza menos memória auxiliar que Insertionsort.

Assinale a alternativa correta.

- A** Apenas I está correta.
- B** I e III estão corretas.
- C** II e IV estão corretas.
- D** I, II e III estão corretas.
- E** Todas estão corretas.

- 49) Sistemas de controle de estoque utilizam arrays para armazenar quantidades de produtos em diferentes depósitos. O acesso direto por índice permite consultas rápidas às quantidades disponíveis.

Considere um array de 10 posições (índices 0 a 9) que armazena a quantidade de produtos em 10 depósitos diferentes. O array está inicializado com os seguintes valores:

[50, 30, 20, 80, 10, 60, 40, 90, 70, 25]

Após executar as seguintes operações:

1. $\text{array}[3] = \text{array}[3] + 20$
2. $\text{array}[7] = \text{array}[7] - 15$
3. $\text{array}[0] = \text{array}[9] * 2$
4. $\text{array}[5] = (\text{array}[1] + \text{array}[2]) / 2$

Qual é o valor armazenado em $\text{array}[5]$?

- A) 25
 - B) 30
 - C) 40
 - D) 50
 - E) 25
- 50) Um sistema de monitoramento de temperaturas registra leituras horárias em um array de 24 posições (uma para cada hora do dia). O sistema precisa identificar padrões nos dados.

Considere o seguinte array de temperaturas:

[22, 21, 20, 19, 18, 19, 21, 23, 25, 27, 29, 30, 31, 30, 28, 26, 24, 23, 22, 21, 20, 19, 18, 17]

Avalie as afirmações a seguir:

- I. A temperatura máxima do dia foi registrada no índice 12 (13^a hora).

II. A temperatura mínima do dia foi registrada no índice 23 (última posição).

III. O array apresenta temperaturas em ordem crescente das 0h às 5h, e em ordem decrescente das 12h às 23h.

IV. A temperatura média das 6 primeiras horas (índices 0 a 5) é menor que a média das 6 últimas horas (índices 18 a 23).

É correto apenas o que se afirma em

- (A) I e II.
- (B) II e III.
- (C) I, II e III.
- (D) I, III e IV.
- (E) II, III e IV.

DISCURSIVAS

- 51 Roteadores de rede precisam gerenciar filas de pacotes para controlar o tráfego entre interfaces. Uma estratégia amplamente adotada é o FIFO (*First In, First Out*): pacotes são enfileirados na ordem de chegada e transmitidos nessa mesma ordem. Em situações de congestionamento, o roteador pode precisar descartar pacotes quando a fila atinge sua capacidade máxima.

Um engenheiro implementou uma fila circular com vetor de tamanho MAX em linguagem C. A estrutura e as operações são definidas abaixo, mas a função `enfileira` está incompleta:

```
#define MAX 6

typedef struct {
    int dados[MAX];
    int inicio, fim, tamanho;
```



```
} Fila;

void inicializa(Fila *f) {
    f->inicio = f->fim = f->tamanho = 0;
}

// Retorna 1 se enfileirou com sucesso, 0 se a fila estava
cheia
int enfileira(Fila *f, int val) {
    if (f->tamanho == MAX) return 0;
    f->dados[f->fim] = val;
    /* LINHA A */
    f->tamanho++;
    return 1;
}

int desenfileira(Fila *f) {
    if (f->tamanho == 0) return -1;
    int val = f->dados[f->inicio];
    f->inicio = (f->inicio + 1) % MAX;
    f->tamanho--;
    return val;
}
```

Com base nas informações apresentadas, faça o que se pede nos itens a seguir.

- a** Escreva a instrução que deve substituir o comentário `/* LINHA A */` para que o avanço do ponteiro `fim` respeite o comportamento circular da fila. Justifique por que o operador módulo é necessário nesse contexto. (valor: 3,0 pontos)
- b** Simule a execução da seguinte sequência de operações sobre uma fila inicializada com `MAX = 6`: `enfileira(10)`, `enfileira(20)`, `enfileira(30)`, `desenfileira()`, `desenfileira()`, `enfileira(40)`, `enfileira(50)`, `enfileira(60)`, `enfileira(70)`, `enfileira(80)`. Ao final, indique os valores de `inicio`, `fim` e `tamanho`, e apresente o conteúdo do vetor `dados`. (valor: 3,0 pontos)
- c** Explique a diferença entre uma fila implementada com vetor simples (não circular) e uma fila circular em relação ao aproveitamento de espaço de memória. Em seguida, descreva uma situação em que

o uso da fila circular é imprescindível para evitar desperdício de memória. (valor: 4,0 pontos)

- 52) Uma empresa de logística precisa ordenar, a cada fechamento de turno, um vetor com os identificadores numéricos de milhares de entregas realizadas no dia. A equipe de TI avalia dois algoritmos de ordenação baseados em comparação — *mergesort* e *quicksort* — antes de decidir qual adotar em produção. Ambos pertencem à classe dos algoritmos de divisão e conquista, mas diferem em garantias de desempenho, uso de memória auxiliar e comportamento em casos extremos.

Considere as implementações canônicas de ambos os algoritmos para ordenar um vetor de n inteiros, sem otimizações adicionais (pivô sempre o último elemento no *quicksort*).

Com base nas informações apresentadas, faça o que se pede nos itens a seguir.

- a) Apresente a relação de recorrência do *mergesort* e resolva-a utilizando o Teorema Mestre, indicando explicitamente qual caso do teorema se aplica. Em seguida, escreva a complexidade de tempo no melhor caso, no caso médio e no pior caso do *mergesort*. (valor: 3,0 pontos)
- b) Descreva o pior caso do *quicksort* com pivô no último elemento e indique qual entrada o provoca. Escreva a relação de recorrência correspondente e sua solução assintótica. Explique como a escolha de pivô por mediana de três elementos mitiga esse problema. (valor: 3,0 pontos)
- c) Escreva em linguagem C a função `merge`, responsável por intercalar dois subvetores ordenados `v[ini..meio]` e `v[meio+1..fim]` em um único vetor ordenado. Sua implementação deve utilizar um vetor auxiliar e ter complexidade $\Theta(n)$ no número de elementos intercalados. (valor: 4,0 pontos)